Annexe 17.4/17.5/17.6 - Scripts SQL livrés (générés depuis les fichiers du dépôt)

Fichiers sources : `sql/01_minishop_schema.sql` (DDL + données), `sql/02_minishop_procedures.sql` (17 procédures + 1 fonction), `sql/03_minishop_triggers.sql` (16 déclencheurs), `sql/04_minishop_demo.sql` (commandes de démonstration créées **par les procédures**, donc avec triggers actifs), `tests/perf/mesurer.sh` (protocole de mesure `ENF-01`) et `scripts/gen_volumes.sh` (jeu de mesure de volumétrie).

(A->F : scripts SQL, fixture et manifeste de tests ; G->H : mesures de performance et volumétrie ; I->J : CI et fichiers hors dépôt.) Ils sont reproduits ici intégralement **depuis les fichiers réels** pour que le document rendu soit autoportant ; le dépôt reste la source de vérité (les régénérer : `bash "$DIR_SCRIPTS/build-docs.sh"`).

A. Script de création et de peuplement - `sql/01_minishop_schema.sql`

(309 lignes, copié verbatim depuis le dépôt.)

```
-- ================================================================
-- MiniShop - 01 : script de création de la base de données (DDL + données)
-- ----------------------------------------------------------------
-- SAE - BUT Informatique / L3 - groupe de 3 étudiants
-- SGBD cible  : MySQL 8.0.16 ou plus récent (les CHECK sont appliqués
--               depuis la 8.0.16) - testé aussi sur MariaDB 11.8.
-- Charset     : utf8mb4 / utf8mb4_unicode_ci (identique MySQL et MariaDB)
-- Moteur      : InnoDB (transactions, clés étrangères, verrouillage de lignes)
--
-- Chargement (depuis la racine du dépôt) :
--   mysql -u root -p < sql/01_minishop_schema.sql
--   mysql -u root -p < sql/02_minishop_procedures.sql
--   mysql -u root -p < sql/03_minishop_triggers.sql
--
-- Ou, via PHP PDO (script public/install.php) : exécuter les 3 fichiers
-- dans cet ordre (le fichier 02 contient des DELIMITER, donc il faut le
-- client mysql ou un parseur dédié ; en PDO, copier le corps des routines
-- sans la ligne DELIMITER).
--
-- Conventions de nommage (règle CT-nommage) :
--   tables      MAJUSCULES au pluriel        CLIENT, PRODUIT, LIGNE_COMMANDE
--   clés primaires id_<table-singulier>      id_client, id_produit
--   clés étrangères  <table-référencée>_id    id_commande (fk vers COMMANDE)
--   index          idx_*, uniques uk_*, FK fk_*
--
-- Le document « Spécification du système » §12 et le document
-- « Conception de BD » §2 décrivent la justification de chaque choix.
-- ================================================================

-- (le harnais scripts/load_db.sh cree la base ; le USE ci-dessous rend le
--  fichier autonome lorsqu'il est joue directement : mysql < sql/01...sql)
DROP DATABASE IF EXISTS minishop;   -- à retirer si la base est partagée
```

```
CREATE DATABASE IF NOT EXISTS minishop CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE minishop;
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;           -- permet un rejeu idempotent du script

-- ------------------------------------------------------------------------
-- Table ADMINISTRATEUR  (RB-13 : seul un compte du Back-office gère le catalogue)
-- ------------------------------------------------------------------------
CREATE TABLE administrateur (
  id_admin            INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nom                 VARCHAR(100) NOT NULL,
  prenom              VARCHAR(100) NOT NULL,
  email               VARCHAR(190) NOT NULL,
  mot_de_passe_hash   CHAR(60)     NOT NULL COMMENT 'sortie de password_hash() : jamais de mot de
passe en clair (RB-12)',
  role                ENUM('SUPER','GESTIONNAIRE') NOT NULL DEFAULT 'GESTIONNAIRE'
                      COMMENT 'SUPER = peut créer/bloquer des comptes admins',
  actif               TINYINT(1)   NOT NULL DEFAULT 1,
  date_creation       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  derniere_connexion  DATETIME     NULL,
  CONSTRAINT pk_administrateur PRIMARY KEY (id_admin),
  CONSTRAINT uk_admin_email    UNIQUE KEY (email),
  CONSTRAINT ck_admin_actif    CHECK (actif IN (0,1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Personnel du Back-office : produits, catégories, stocks, commandes';

-- ------------------------------------------------------------------------
-- Table CLIENT
-- RB-01 : l'email est unique (contrainte uk_client_email)
-- ------------------------------------------------------------------------
CREATE TABLE client (
  id_client           INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nom                 VARCHAR(100) NOT NULL,
  prenom              VARCHAR(100) NOT NULL,
  email               VARCHAR(190) NOT NULL,
  mot_de_passe_hash   CHAR(60)     NOT NULL COMMENT 'password_hash(PASSWORD_BCRYPT) ; +10 si re-
hash ultérieur',
  telephone           VARCHAR(20)  NULL,
  adresse_livraison   VARCHAR(255) NULL,
  code_postal         VARCHAR(10)  NULL,
  ville               VARCHAR(100) NULL,
  actif               TINYINT(1)   NOT NULL DEFAULT 1,
  date_creation       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  derniere_connexion  DATETIME     NULL,
  CONSTRAINT pk_client        PRIMARY KEY (id_client),
  CONSTRAINT uk_client_email  UNIQUE KEY (email),
  CONSTRAINT ck_client_actif  CHECK (actif IN (0,1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Client inscrit : panier, commandes, historique';

-- ------------------------------------------------------------------------
-- Table CATEGORIE
-- RB-07 / RB-08 : 1 catégorie -> N produits ; 1 produit -> exactement 1 catégorie
-- ------------------------------------------------------------------------
CREATE TABLE categorie (
  id_categorie      INT UNSIGNED NOT NULL AUTO_INCREMENT,
```

```
  nom               VARCHAR(100) NOT NULL,
  slug              VARCHAR(120) NOT NULL,
  description       VARCHAR(500) NULL,
  CONSTRAINT pk_categorie       PRIMARY KEY (id_categorie),
  CONSTRAINT uk_categorie_nom   UNIQUE KEY (nom),
  CONSTRAINT uk_categorie_slug  UNIQUE KEY (slug)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Rayon du catalogue';

-- ---------------------------------------------------------------------
-- Table PRODUIT
-- RB-02 : prix strictement positif      (ck_produit_prix, défaut NOT NULL)
-- RB-03 : stock jamais négatif          (ck_produit_stock + trg_produit_stock_non_negatif)
-- RB-14 : référence et slug uniques
-- ---------------------------------------------------------------------
CREATE TABLE produit (
  id_produit       INT UNSIGNED NOT NULL AUTO_INCREMENT,
  reference        VARCHAR(30)  NOT NULL,
  nom              VARCHAR(150) NOT NULL,
  slug             VARCHAR(180) NOT NULL,
  description      TEXT         NULL,
  prix_ht          DECIMAL(10,2) NOT NULL,
  tva              DECIMAL(5,2)  NOT NULL DEFAULT 20.00,
  prix_ttc         DECIMAL(10,2) NOT NULL
                   COMMENT 'généré par trg_produit_ttc = ROUND(prix_ht*(1+tva/100),2) ; jamais
écrit par l''application',
  stock            INT          NOT NULL DEFAULT 0 COMMENT 'quantité physique disponible (RB-03)',

  stock_initial  INT          NOT NULL DEFAULT -1 COMMENT 'quantite du seed : etat de reference de
 la fixture de test (-1 = pas encore renseigne ; alimente par trg_produit_ttc)',
  seuil_alerte     INT          NOT NULL DEFAULT 3 COMMENT 'dessous : produit en alerte (EF-
ADM-05)',
  id_categorie     INT UNSIGNED NOT NULL COMMENT 'RB-07 : une seule catégorie par produit',
  visible          TINYINT(1)   NOT NULL DEFAULT 1 COMMENT '1 = visible côté client',
  image_url        VARCHAR(255) NULL,
  date_creation    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  date_modification DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_produit          PRIMARY KEY (id_produit),
  CONSTRAINT fk_produit_categorie FOREIGN KEY (id_categorie)
        REFERENCES categorie (id_categorie) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT uk_produit_reference UNIQUE KEY (reference),
  CONSTRAINT uk_produit_slug      UNIQUE KEY (slug),
  CONSTRAINT ck_produit_prix      CHECK (prix_ht > 0),
  CONSTRAINT ck_produit_stock     CHECK (stock >= 0),
  CONSTRAINT ck_produit_visible   CHECK (visible IN (0,1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Article du catalogue (RB-02, RB-03, RB-07, RB-14)';

-- Ligne ajoutée après coup pour alléger la création de table (fulltext recherche EF-VIS-02)
ALTER TABLE produit ADD FULLTEXT KEY ft_produit_recherche (nom, description);
CREATE INDEX idx_produit_categorie ON produit (id_categorie);
CREATE INDEX idx_produit_visible    ON produit (visible, stock);

-- ---------------------------------------------------------------------
-- Table COMMANDE
-- RB-10 : 1 commande -> 1 seul client (id_client NOT NULL)
```

```
-- RB-11 : statut contraint par l'ENUM + la matrice de transitions
-- RB-15 : montant_total est une somme calculée, jamais une saisie
-- ------------------------------------------------------------------
-- ------------------------------------------------------------------
-- Table de parametres metier : les regles chiffrees (frais de port, seuil de
-- franchise, TVA par defaut, durees de conservation) sont des DONNEES et non
-- des constantes recopiees dans le code PHP : c'est ce qui permet de les faire
-- evoluer sans toucher a une ligne de SQL ni a une ligne de PHP, et de les
-- tracer. Les procedures les lisent via fn_param() (RB-15, ENF-14).
-- ------------------------------------------------------------------
DROP TABLE IF EXISTS parametre;
CREATE TABLE parametre (
  cle     VARCHAR(50)  NOT NULL,
  valeur  VARCHAR(100) NOT NULL,
  unite   VARCHAR(10)  NULL COMMENT 'EUR, %, mois, jours...',
  comment VARCHAR(255) NULL,
  date_modification DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_parametre PRIMARY KEY (cle)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

INSERT INTO parametre (cle, valeur, unite, comment) VALUES
  ('frais_port',            '4.90',  'EUR TTC', 'Livraison standard, montant affiche avant la
validation (art. L221-5 4 deg C. conso.)'),
  ('franchise_port',        '80.00', 'EUR TTC', 'Port offert au-dela de ce montant de
marchandises'),
  ('tva_defaut',            '20.00', '%',       'Taux par defaut propose a la creation d un
produit'),
  ('conservation_compte',   '36',    'mois',    'Donnees client en base active apres le dernier
contact (referentiel CNIL)');

CREATE TABLE commande (
  id_commande      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  numero           VARCHAR(20)  NOT NULL COMMENT 'numéro lisible CMD2026-000123',
  id_client        INT UNSIGNED NOT NULL,
  adresse_livraison VARCHAR(255) NOT NULL,
  statut           ENUM('BROUILLON','EN_PREPARATION','PAYEE','EXPEDIEE','LIVREE','ANNULEE')
                   NOT NULL DEFAULT 'BROUILLON',
  montant_total    DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  frais_port       DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT 'somme figee au snapshot, pas une
saisie (RB-15)',
  commentaire      VARCHAR(500) NULL,
  date_commande    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  date_modification DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_commande       PRIMARY KEY (id_commande),
  CONSTRAINT uk_commande_numero UNIQUE KEY (numero),
  CONSTRAINT fk_commande_client FOREIGN KEY (id_client)
        REFERENCES client (id_client) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT ck_commande_montant CHECK (montant_total >= 0),
  CONSTRAINT ck_commande_port    CHECK (frais_port >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Commande d''un client (RB-10, RB-11, RB-15)';

CREATE INDEX idx_commande_client ON commande (id_client);
CREATE INDEX idx_commande_statut ON commande (statut);

-- ------------------------------------------------------------------
```

```
-- Table LIGNE_COMMANDE
-- RB-04 : au moins une ligne (contrôle dans sp_confirm_order + trg_commande_delete)
-- RB-05 : quantité > 0 (ck + défaut NOT NULL)
-- RB-06 : prix_unitaire = prix TTC AU MOMENT DE L'ACHAT (figé par trg_ligne_prix_snapshot)
-- ------------------------------------------------------------------
CREATE TABLE ligne_commande (
  id_ligne        BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_commande     INT UNSIGNED NOT NULL,
  id_produit      INT UNSIGNED NOT NULL,
  quantite        INT          NOT NULL,
  prix_unitaire   DECIMAL(10,2) NOT NULL COMMENT 'RB-06 : snapshot du prix au moment de l''achat',
  total_ligne     DECIMAL(10,2) NOT NULL COMMENT 'généré = quantite * prix_unitaire',
  CONSTRAINT pk_ligne_commande PRIMARY KEY (id_ligne),
  CONSTRAINT fk_ligne_commande FOREIGN KEY (id_commande)
        REFERENCES commande (id_commande) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_ligne_produit  FOREIGN KEY (id_produit)
        REFERENCES produit (id_produit) ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT ck_ligne_quantite CHECK (quantite > 0),
  CONSTRAINT ck_ligne_prix     CHECK (prix_unitaire > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Détail d''une commande ; produit fige a l instant T';

CREATE INDEX idx_ligne_commande ON ligne_commande (id_commande);
CREATE INDEX idx_ligne_produit  ON ligne_commande (id_produit);

-- ------------------------------------------------------------------
-- Table ORDER_STATUS_HISTORY
-- Historique des changements de statut, alimenté UNIQUEMENT par le
-- trigger trg_history_statut (personne ne peut « oublier » de tracer).
-- ------------------------------------------------------------------
CREATE TABLE order_status_history (
  id            BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_id      INT UNSIGNED NOT NULL,
  old_status    VARCHAR(20)  NULL,
  new_status    VARCHAR(20)  NOT NULL,
  changed_by    INT UNSIGNED NULL COMMENT 'id_client si le client annule, sinon id_admin',
  changed_by_role ENUM('CLIENT','ADMIN','SYSTEME') NOT NULL DEFAULT 'SYSTEME',
  commentaire   VARCHAR(500) NULL,
  changed_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_history PRIMARY KEY (id),
  CONSTRAINT fk_history_commande FOREIGN KEY (order_id)
        REFERENCES commande (id_commande) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Piste d''audit des statuts (RB-11) - écriture automatique par trigger';

-- ------------------------------------------------------------------
-- FIN DES TABLES - insertion du jeu de données de démonstration
-- (mot de passe des comptes de test : Demo2026! pour les clients,
--  Admin2026! pour le back-office - hash générés par password_hash()).
-- ------------------------------------------------------------------
INSERT INTO administrateur (nom, prenom, email, mot_de_passe_hash, role) VALUES
  ('Nguyen','Alice','admin@minishop.fr','$2b$10$ASp02qmyg5r3VvXRIV60/ObRO3IDB6bJ7DWgH20FOECvoNtDPN
NW2','SUPER');

INSERT INTO categorie (nom, slug, description) VALUES
  ('Informatique','informatique','Ordinateurs portables, périphériques et composants'),
```

```
  ('Audio','audio','Casques, enceintes et microphones'),
  ('Accessoires','accessoires','Sacs, câbles, supports et batteries'),
  ('Objets connectés','objets-connectes','Montres et bracelets connectés');

-- id_categorie : 1 Informatique, 2 Audio, 3 Accessoires, 4 Objets connectés
-- prix_ttc est posé ici volontairement : le trigger trg_produit_ttc le
-- recalcule à chaque écriture suivante (il sert surtout en UPDATE).
INSERT INTO produit
  (reference, nom, slug, description, prix_ht, tva, prix_ttc, stock, seuil_alerte, id_categorie,
visible, image_url) VALUES
  ('PC-PORT-001','Ordinateur portable Zen 14','ordinateur-portable-zen-14','Écran IPS 14", 16 Go
de RAM, SSD 512 Go, autonomie 12 h.',699.00,20.00,838.80,12,3,1,1,'img/zen14.jpg'),
  ('PC-FIXE-002','Station de travail Turbo X','station-de-travail-turbo-x','Boîtier compact, 8
cœurs, 32 Go de RAM, SSD NVMe 1 To.',1249.00,20.00,1498.80,4,2,1,1,'img/turbox.jpg'),
  ('Ecran-27-003','Écran 27" QHD','ecran-27-qhd','Dalle IPS QHD 75 Hz, HDR10, pied
réglable.',179.90,20.00,215.88,25,5,1,1,'img/qhd27.jpg'),
  ('CLAV-004','Clavier mécanique K87','clavier-mecanique-k87','Switches tactiles, rétroéclairage,
sans fil 2,4 GHz.',79.00,20.00,94.80,40,10,1,1,'img/k87.jpg'),
  ('CASQ-005','Casque sans fil Aura','casque-sans-fil-aura','Réduction de bruit active, 35 h
d''autonomie.',129.00,20.00,154.80,18,5,2,1,'img/aura.jpg'),
  ('ENCH-006','Enceinte portable Boom','enceinte-portable-boom','IPX7, 20 h d''autonomie, stéréo
TrueWireless.',59.00,20.00,70.80,30,8,2,1,'img/boom.jpg'),
  ('MICRO-007','Micro studio ProCast','micro-studio-procast','Condensateur cardioïde, bras
articulé inclus.',99.00,20.00,118.80,2,1,2,1,'img/procast.jpg'),
  ('SAC-008','Sac à dos 25 L nomade','sac-a-dos-25-l-nomade','Compartiment 16", tissu déperlant,
port USB.',49.00,20.00,58.80,22,6,3,1,'img/sac25.jpg'),
  ('CABLE-009','Cable USB-C 2 m renforcé','cable-usb-c-2-m-renforce','Charge 100 W, transfert 10
Gbps, tresse nylon.',12.50,20.00,15.00,150,30,3,1,'img/usbc.jpg'),
  ('POWER-010','Batterie externe 20 000 mAh','batterie-externe-20-000-mah','Charge rapide 65 W,
deux ports USB-C.',45.00,20.00,54.00,0,5,3,1,'img/power20.jpg'),
  ('MONTRE-011','Montre connectée Fit 2','montre-connectee-fit-2','Cardio, GPS, 14 jours
d''autonomie, étanche 5 ATM.',149.00,20.00,178.80,9,3,4,1,'img/fit2.jpg'),
  ('BRAC-012','Bracelet connecté Band','bracelet-connecte-band','Suivi du sommeil et des
notifications, 10 jours.',35.00,20.00,42.00,60,15,4,0,'img/band.jpg');

INSERT INTO client (nom, prenom, email, mot_de_passe_hash, telephone, adresse_livraison,
code_postal, ville) VALUES
  ('Dupont','Alice','alice@example.com','$2b$10$ldtyUy7EyQ4YJ0L8qBWZxuTuefzalZmCDOnXNnBHHZNm8cZqOe
rjO','0612345678','12 rue de France','06000','Nice'),
  ('Martin','Bruno','bruno@example.com','$2b$10$ldtyUy7EyQ4YJ0L8qBWZxuTuefzalZmCDOnXNnBHHZNm8cZqOe
rjO','0698765432','5 avenue Jean Médecin','06000','Nice'),
  ('Moretti','Carla','carla@example.com','$2b$10$ldtyUy7EyQ4YJ0L8qBWZxuTuefzalZmCDOnXNnBHHZNm8cZqO
erjO',NULL,'8 rue Papin','06300','Nice');

SET FOREIGN_KEY_CHECKS = 1;

-- ---------------------------------------------------------------------
-- VUES : une seule définition des formules partagées par le front-office,
-- le back-office et la procédure sp_search_products (évite la divergence
-- de calcul entre PHP et SQL, cf. RB-16 / RB-19 / ENF-13).
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_etat_stock AS
SELECT p.id_produit AS id_produit, p.reference AS reference, p.nom AS nom, p.stock AS stock,
p.seuil_alerte AS seuil_alerte,
       CASE WHEN p.stock = 0                    THEN 'RUPTURE'
            WHEN p.stock <= p.seuil_alerte       THEN 'TRES_BAS'
```

```
            ELSE 'DISPONIBLE' END AS etat
  FROM produit p;

CREATE OR REPLACE VIEW v_catalogue AS
SELECT p.id_produit, p.reference, p.slug, p.nom, p.description, p.prix_ht, p.tva,
       p.prix_ttc, p.stock, p.visible, p.image_url,
       c.id_categorie, c.nom AS categorie, c.slug AS categorie_slug,
       e.etat
  FROM produit p
  JOIN categorie c   ON c.id_categorie = p.id_categorie
  JOIN v_etat_stock e ON e.id_produit  = p.id_produit
 WHERE p.visible = 1;

CREATE OR REPLACE VIEW v_commandes_client AS
SELECT o.id_commande, o.numero, o.statut, o.montant_total, o.frais_port, o.adresse_livraison,
       o.date_commande, o.id_client,
       CONCAT(cl.prenom, ' ', cl.nom) AS client,
       (SELECT COUNT(*) FROM ligne_commande l WHERE l.id_commande = o.id_commande) AS nb_lignes
  FROM commande o
  JOIN client cl ON cl.id_client = o.id_client;

-- ======================================================================
-- Historique de l'application : les commandes de démonstration sont
-- créées par les procédures stockées (voir sql/04_minishop_demo_orders.sql)
-- afin que les triggers fassent réellement sonner le dispositif.
-- ======================================================================
```

B. Procédures stockées - `sql/02_minishop_procedures.sql`

(837 lignes, copié verbatim depuis le dépôt.)

```
-- ======================================================================
-- MiniShop - 02 : procédures stockées (17 procédures + 1 fonction, minimum demandé : 5)
-- ----------------------------------------------------------------------
-- Conventions :
--   * Toute erreur métier est remontée par  SIGNAL SQLSTATE '45000'
--     avec un MESSAGE_TEXT court, stable, préfixé par une clé (CODE_RAISON)
--     que l'application traduit en message utilisateur.
--   * Les procédures qui écrivent plusieurs lignes sont transactionnelles
--     (START TRANSACTION / COMMIT / ROLLBACK) : c'est l'un des arguments
--     pédagogiques majeurs du choix « procédure stockée plutôt que PHP ».
--   * Les procédures ne renvoient JAMAIS le hash de mot de passe à l'IHM :
--     la couche modèle le compare avec password_verify() (RB-12).
--
-- Chargement :  mysql -u root -p minishop < sql/02_minishop_procedures.sql
-- ======================================================================

-- Nom de la base : passe au client (`mariadb minishop < fichier`) par
-- scripts/load_db.sh ; en mode interactif, taper `USE minishop;` d'abord.

-- ----------------------------------------------------------------------
-- 1. sp_create_account - création d'un compte client (RB-01 : email unique)
--    Retourne id_client ; p_code_retour = 'EMAIL_DEJA_UTILISE' si doublon.
-- ----------------------------------------------------------------------
```

```
-- ----------------------------------------------------------------------
-- 0. Lecteur unique des parametres metier (RB-15, ENF-14) : une fonction,
--    pas une constante recopiee dans chaque procedural et dans le PHP.
--    CREATE FUNCTION exige SUPER sur certains SGBD : si la fonction ne peut
--    pas etre creee, sp_compute_shipping utilise la valeur par defaut documentee.
-- ----------------------------------------------------------------------
DROP FUNCTION IF EXISTS fn_param $$
CREATE FUNCTION fn_param (p_cle VARCHAR(50), p_defaut VARCHAR(100))
RETURNS VARCHAR(100)
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE v VARCHAR(100);
  SELECT valeur INTO v FROM parametre WHERE cle = p_cle LIMIT 1;
  RETURN IFNULL(v, p_defaut);
END $$

-- ----------------------------------------------------------------------
-- 0 bis. sp_compute_shipping - regle de frais de port (ENF-14, art. L221-5
--        du code de la consommation et art. 5(1)(e) de la directive 2011/83/UE :
--        le port doit etre affiche avant la validation, donc calcule AVANT).
--        Regle : 4,90 EUR TTC en standard, offert a partir de 80,00 EUR.
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_compute_shipping $$
CREATE PROCEDURE sp_compute_shipping (
  IN  p_marchandises DECIMAL(10,2),
  OUT p_frais_port   DECIMAL(10,2),
  OUT p_motif        VARCHAR(60)
)
BEGIN
  DECLARE v_port    DECIMAL(10,2);
  DECLARE v_franchise DECIMAL(10,2);
  SET v_port     = CAST(fn_param('frais_port', '4.90') AS DECIMAL(10,2));
  SET v_franchise = CAST(fn_param('franchise_port', '80.00') AS DECIMAL(10,2));
  IF p_marchandises >= v_franchise THEN
    SET p_frais_port = 0.00;
    SET p_motif = CONCAT('PORT_OFFERT_A_PARTIR_DE_', v_franchise);
  ELSE
    SET p_frais_port = v_port;
    SET p_motif = 'LIVRAISON_STANDARD';
  END IF;
END $$

DROP PROCEDURE IF EXISTS sp_create_account $$
CREATE PROCEDURE sp_create_account (
  IN  p_nom         VARCHAR(100),
  IN  p_prenom      VARCHAR(100),
  IN  p_email       VARCHAR(190),
  IN  p_hash        CHAR(60),
  OUT p_id_client   INT UNSIGNED,
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  IF p_nom IS NULL OR p_prenom IS NULL OR TRIM(p_nom) = '' OR TRIM(p_prenom) = '' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CHAMPS_OBLIGATOIRES';
  END IF;
```

```
  IF p_hash IS NULL OR CHAR_LENGTH(p_hash) < 60 THEN
    -- garde-fou : on refuse une chaîne qui ne ressemble pas à une sortie de password_hash()
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'HASH_MOT_DE_PASSE_INVALIDE';
  END IF;

  SET p_id_client = NULL;
  IF EXISTS (SELECT 1 FROM client WHERE email = p_email) THEN
    SET p_code_retour = 'EMAIL_DEJA_UTILISE';
  ELSE
    INSERT INTO client (nom, prenom, email, mot_de_passe_hash)
      VALUES (p_nom, p_prenom, p_email, p_hash);
    SET p_id_client  = LAST_INSERT_ID();
    SET p_code_retour = 'OK';
  END IF;
END $$

-- ----------------------------------------------------------------------
-- 2. sp_get_credentials - récupération de l'identifiant + hash pour la
--    connexion. Le compte inactif est refusé en base, pas seulement en PHP.
--    LA COMPARAISON DU MOT DE PASSE RESTE EN PHP : seul password_verify()
--    sait comparer un bcrypt en temps constant (RB-12, SEC-04).
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_get_credentials $$
CREATE PROCEDURE sp_get_credentials (
  IN  p_email     VARCHAR(190),
  OUT p_id_client INT UNSIGNED,
  OUT p_hash      CHAR(60),
  OUT p_statut    VARCHAR(20)
)
BEGIN
  SELECT id_client, mot_de_passe_hash, IF(actif = 1, 'OK', 'COMPTE_BLOQUE')
    INTO p_id_client, p_hash, p_statut
    FROM client
   WHERE email = p_email;

  IF p_id_client IS NULL THEN
    -- message volontairement identique à « mot de passe erroné » côté IHM (SEC-05)
    SET p_hash   = NULL;
    SET p_statut = 'INCONNU';
  END IF;
END $$

-- ----------------------------------------------------------------------
-- 3. sp_update_client - modification de ses propres informations
--    (EF-CLI-02). Le changement d'email ré-active la règle RB-01.
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_update_client $$
CREATE PROCEDURE sp_update_client (
  IN  p_id_client   INT UNSIGNED,
  IN  p_nom         VARCHAR(100),
  IN  p_prenom      VARCHAR(100),
  IN  p_email       VARCHAR(190),
  IN  p_telephone   VARCHAR(20),
  IN  p_adresse     VARCHAR(255),
  IN  p_code_postal VARCHAR(10),
  IN  p_ville       VARCHAR(100),
```

```
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  IF NOT EXISTS (SELECT 1 FROM client WHERE id_client = p_id_client) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CLIENT_INTROUVABLE';
  END IF;

  IF EXISTS (SELECT 1 FROM client WHERE email = p_email AND id_client <> p_id_client) THEN
    SET p_code_retour = 'EMAIL_DEJA_UTILISE';
  ELSE
    UPDATE client
       SET nom = p_nom, prenom = p_prenom, email = p_email,
           telephone = p_telephone, adresse_livraison = p_adresse,
           code_postal = p_code_postal, ville = p_ville
     WHERE id_client = p_id_client;
    SET p_code_retour = 'OK';
  END IF;
END $$

-- ---------------------------------------------------------------------
-- 4. sp_search_products - recherche + filtres + tri + pagination (EF-VIS-02/03)
--    Toutes les valeurs utilisateur sont injectées dans la requête préparée
--    via des paramètres marqués ; seul le tri provient d'une liste blanche.
--    Le LIKE est construit en SQL (CONCAT), jamais en PHP (SEC-01).
-- ---------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_search_products $$
CREATE PROCEDURE sp_search_products (
  IN p_mot_cle      VARCHAR(120),
  IN p_id_categorie INT UNSIGNED,
  IN p_prix_min     DECIMAL(10,2),
  IN p_prix_max     DECIMAL(10,2),
  IN p_en_stock     TINYINT,
  IN p_trie_par     VARCHAR(20),
  IN p_page         INT,
  IN p_par_page     INT,
  IN p_admin        TINYINT
)
BEGIN
  -- Technique « tous les filtres sont toujours lies » : le texte SQL est
  -- dynamique (seule la partie necessaire est construite) mais le NOMBRE de
  -- marqueurs positionnels (?) reste constant, ce qui rend EXECUTE ... USING
  -- portable (MySQL comme MariaDB n'acceptent que des ? en requete preparee).
  -- Les valeurs n'entrent JAMAIS dans le texte de la requete : elles passent
  -- par des marqueurs positionnels (?) prepares (SEC-01).
  DECLARE v_kw   VARCHAR(120);
  DECLARE v_cat  INT UNSIGNED;
  DECLARE v_min  DECIMAL(10,2);
  DECLARE v_max  DECIMAL(12,2);
  DECLARE v_ok   TINYINT;
  DECLARE v_off  INT;
  DECLARE v_lim  INT;

  SET v_kw  = IFNULL(NULLIF(TRIM(p_mot_cle), ''), '');
  SET v_cat = IFNULL(p_id_categorie, 0);
  SET v_min = IFNULL(p_prix_min, 0);
  SET v_max = IFNULL(p_prix_max, 99999999.99);  -- DECIMAL(12,2) : borne haute « tout accepté »
```

```
  SET v_ok  = IFNULL(p_en_stock, 0);
  SET v_lim = LEAST(GREATEST(IFNULL(NULLIF(p_par_page, 0), 12), 1), 60);
  SET v_off = (GREATEST(IFNULL(NULLIF(p_page, 0), 1), 1) - 1) * v_lim;

  SET @sql = CONCAT(
    'SELECT p.id_produit, p.reference, p.slug, p.nom, p.prix_ttc, p.stock, p.visible,
            c.nom AS categorie,
            CASE WHEN p.stock = 0 THEN "RUPTURE"
                 WHEN p.stock <= p.seuil_alerte THEN "TRES_BAS"
                 ELSE "DISPONIBLE" END AS etat_stock
       FROM produit p
       JOIN categorie c ON c.id_categorie = p.id_categorie
      WHERE (? = 0 OR p.visible = 1)
        AND (? = "" OR p.nom LIKE CONCAT("%", ?, "%")
             OR p.description LIKE CONCAT("%", ?, "%")
             OR p.reference LIKE CONCAT("%", ?, "%"))
        AND (? = 0 OR p.id_categorie = ?)
        AND p.prix_ttc >= ?
        AND p.prix_ttc <= ?
        AND (? = 0 OR p.stock > 0)',
    CASE
      WHEN p_trie_par = 'prix_asc'  THEN ' ORDER BY p.prix_ttc ASC, p.id_produit'
      WHEN p_trie_par = 'prix_desc' THEN ' ORDER BY p.prix_ttc DESC, p.id_produit'
      WHEN p_trie_par = 'nouveaute' THEN ' ORDER BY p.date_creation DESC, p.id_produit'
      WHEN p_trie_par = 'stock'     THEN ' ORDER BY p.stock DESC, p.id_produit'
      ELSE ' ORDER BY p.nom ASC'
    END,
    ' LIMIT ? OFFSET ?');

  -- ordre des marqueurs = ordre des ? dans le texte de la requete
  SET @f_visible = IF(IFNULL(p_admin, 0) = 1, 0, 1);
  SET @f_kw  = v_kw, @f_cat = v_cat, @f_min = v_min, @f_max = v_max,
      @f_ok  = v_ok, @f_lim = v_lim, @f_off = v_off;

  SET @s = @sql;
  PREPARE stmt FROM @s;
  EXECUTE stmt USING @f_visible, @f_kw, @f_kw, @f_kw, @f_kw, @f_cat, @f_cat, @f_min, @f_max,
@f_ok, @f_lim, @f_off;
  DEALLOCATE PREPARE stmt;

  -- second jeu de resultats : le nombre total de produits trouves (pagination)
  SET @s = 'SELECT COUNT(*) AS total_trouves
              FROM produit p
             WHERE (? = 0 OR p.visible = 1)
               AND (? = "" OR p.nom LIKE CONCAT("%", ?, "%")
                    OR p.description LIKE CONCAT("%", ?, "%")
                    OR p.reference LIKE CONCAT("%", ?, "%"))
               AND (? = 0 OR p.id_categorie = ?)
               AND p.prix_ttc >= ?
               AND p.prix_ttc <= ?
               AND (? = 0 OR p.stock > 0)';
  PREPARE stmt FROM @s;
  EXECUTE stmt USING @f_visible, @f_kw, @f_kw, @f_kw, @f_kw, @f_cat, @f_cat, @f_min, @f_max,
@f_ok;
  DEALLOCATE PREPARE stmt;
END $$
```

```
-- ----------------------------------------------------------------------
-- 5. sp_save_product - création / modification côté Back-office
--    (EF-ADM-01, EF-ADM-02). p_id_produit NULL => création.
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_save_product $$
CREATE PROCEDURE sp_save_product (
  IN  p_id_produit   INT UNSIGNED,
  IN  p_reference    VARCHAR(30),
  IN  p_nom          VARCHAR(150),
  IN  p_slug         VARCHAR(180),
  IN  p_description  TEXT,
  IN  p_prix_ht      DECIMAL(10,2),
  IN  p_tva          DECIMAL(5,2),
  IN  p_stock        INT,
  IN  p_seuil_alerte INT,
  IN  p_id_categorie INT UNSIGNED,
  IN  p_visible      TINYINT,
  IN  p_image_url    VARCHAR(255),
  OUT p_id_resultat  INT UNSIGNED,
  OUT p_code_retour  VARCHAR(40)
)
BEGIN
  DECLARE v_dup INT DEFAULT 0;

  IF p_prix_ht IS NULL OR p_prix_ht <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB02_PRIX_DOIT_ETRE_STRICTEMENT_POSITIF';
  END IF;
  IF p_stock IS NULL OR p_stock < 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF';
  END IF;
  IF p_nom IS NULL OR TRIM(p_nom) = '' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'NOM_REQUIS';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM categorie WHERE id_categorie = p_id_categorie) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CATEGORIE_INTROUVABLE';
  END IF;
  IF p_slug IS NULL OR TRIM(p_slug) = '' THEN
    SET p_slug = CONCAT('produit-', IFNULL(p_id_produit, 0));
  END IF;

  SELECT COUNT(*) INTO v_dup FROM produit
   WHERE reference = p_reference AND (id_produit <> IFNULL(p_id_produit, 0));
  IF v_dup > 0 THEN
    SET p_id_resultat = NULL; SET p_code_retour = 'REFERENCE_DEJA_UTILISEE';
  ELSE
    SELECT COUNT(*) INTO v_dup FROM produit
     WHERE slug = p_slug AND (id_produit <> IFNULL(p_id_produit, 0));
    IF v_dup > 0 THEN
      SET p_id_resultat = NULL; SET p_code_retour = 'SLUG_DEJA_UTILISE';
    ELSEIF p_id_produit IS NULL THEN
      INSERT INTO produit (reference, nom, slug, description, prix_ht, tva, prix_ttc,
                           stock, seuil_alerte, id_categorie, visible, image_url)
        VALUES (p_reference, p_nom, p_slug, p_description, p_prix_ht,
                IFNULL(p_tva, 20.00), ROUND(p_prix_ht * (1 + IFNULL(p_tva, 20.00) / 100), 2),
                p_stock, IFNULL(p_seuil_alerte, 3), p_id_categorie,
```

```
                IFNULL(p_visible, 1), p_image_url);
      SET p_id_resultat  = LAST_INSERT_ID();
      SET p_code_retour  = 'OK';
    ELSE
      UPDATE produit
         SET reference = p_reference, nom = p_nom, slug = p_slug,
             description = p_description, prix_ht = p_prix_ht,
             tva = IFNULL(p_tva, tva), stock = p_stock,
             seuil_alerte = IFNULL(p_seuil_alerte, seuil_alerte),
             id_categorie = p_id_categorie, visible = IFNULL(p_visible, visible),
             image_url = p_image_url
       WHERE id_produit = p_id_produit;
      SET p_id_resultat = p_id_produit;
      SET p_code_retour = IF(ROW_COUNT() = 0, 'AUCUNE_MODIFICATION', 'OK');
    END IF;
  END IF;
END $$

-- ----------------------------------------------------------------------
-- 6. sp_delete_product - suppression protégée (EF-ADM-02 + RB-16)
--    Un produit déjà commandé reste au catalogue (visible = 0) afin de
--    conserver l'historique des lignes de commande.
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_delete_product $$
CREATE PROCEDURE sp_delete_product (IN p_id_produit INT UNSIGNED)
BEGIN
  DECLARE v_lignes INT DEFAULT 0;

  SELECT COUNT(*) INTO v_lignes FROM ligne_commande WHERE id_produit = p_id_produit;
  IF v_lignes > 0 THEN
    UPDATE produit SET visible = 0 WHERE id_produit = p_id_produit;   -- sinon : erreur du trigger
  ELSEIF EXISTS (SELECT 1 FROM produit WHERE id_produit = p_id_produit) THEN
    DELETE FROM produit WHERE id_produit = p_id_produit;
  ELSE
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PRODUIT_INTROUVABLE';
  END IF;
END $$

-- ----------------------------------------------------------------------
-- 7. sp_adjust_stock - réappro / inventaire (EF-ADM-04, RB-03)
--    p_mode : 'SET' (valeur absolue) ou 'DELTA' (mouvement +/-)
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_adjust_stock $$
CREATE PROCEDURE sp_adjust_stock (
  IN  p_id_produit  INT UNSIGNED,
  IN  p_mode        VARCHAR(5),
  IN  p_quantite    INT,
  IN  p_motif       VARCHAR(200),
  OUT p_stock       INT,
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  DECLARE v_new INT DEFAULT 0;

  IF p_quantite IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'QUANTITE_REQUISE';
```

```
  END IF;

  SELECT stock INTO v_new FROM produit WHERE id_produit = p_id_produit FOR UPDATE;
  IF v_new IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PRODUIT_INTROUVABLE';
  END IF;

  IF p_mode = 'DELTA' THEN
    SET v_new = v_new + p_quantite;
  ELSE
    SET v_new = p_quantite;
  END IF;

  IF v_new < 0 THEN
    -- la règle RB-03 est aussi garantie par trg_produit_stock_non_negatif :
    -- cette vérification ne fait qu'offrir un message explicite à l'IHM
    SET p_stock = NULL; SET p_code_retour = 'RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF';
  ELSE
    UPDATE produit SET stock = v_new WHERE id_produit = p_id_produit;
    SET p_stock = v_new;
    SET p_code_retour = 'OK';
  END IF;
END $$

-- ---------------------------------------------------------------------
-- 8. sp_save_category - création / renommage (EF-ADM-03)
-- ---------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_save_category $$
CREATE PROCEDURE sp_save_category (
  IN  p_id_categorie INT UNSIGNED,
  IN  p_nom          VARCHAR(100),
  IN  p_slug         VARCHAR(120),
  IN  p_description  VARCHAR(500),
  OUT p_id_resultat  INT UNSIGNED,
  OUT p_code_retour  VARCHAR(40)
)
BEGIN
  IF p_nom IS NULL OR TRIM(p_nom) = '' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'NOM_REQUIS';
  END IF;
  IF p_slug IS NULL OR TRIM(p_slug) = '' THEN
    SET p_slug = LOWER(REPLACE(TRIM(p_nom), ' ', '-'));
  END IF;

  IF p_id_categorie IS NULL THEN
    INSERT INTO categorie (nom, slug, description) VALUES (p_nom, p_slug, p_description)
      ON DUPLICATE KEY UPDATE slug = VALUES(slug), description = VALUES(description);
    IF EXISTS (SELECT 1 FROM categorie WHERE nom = p_nom AND slug <> p_slug) THEN
      SET p_code_retour = 'NOM_DEJA_UTILISE';
      SET p_id_resultat = NULL;
    ELSE
      SET p_id_resultat = LAST_INSERT_ID();
      SET p_code_retour = 'OK';
    END IF;
  ELSE
    UPDATE categorie SET nom = p_nom, slug = p_slug, description = p_description
```

```
     WHERE id_categorie = p_id_categorie;
    SET p_id_resultat = p_id_categorie;
    SET p_code_retour = IF(ROW_COUNT() = 0, 'CATEGORIE_INTROUVABLE', 'OK');
  END IF;
END $$

-- ----------------------------------------------------------------------
-- 9. sp_delete_category - RB-07 : on ne supprime pas une catégorie occupée
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_delete_category $$
CREATE PROCEDURE sp_delete_category (IN p_id_categorie INT UNSIGNED)
BEGIN
  DECLARE v_nb INT DEFAULT 0;

  SELECT COUNT(*) INTO v_nb FROM produit WHERE id_categorie = p_id_categorie;
  IF v_nb > 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'CATEGORIE_NON_VIDE_REASSIGNER_LES_PRODUITS';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM categorie WHERE id_categorie = p_id_categorie) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CATEGORIE_INTROUVABLE';
  END IF;
  DELETE FROM categorie WHERE id_categorie = p_id_categorie;
END $$

-- ----------------------------------------------------------------------
-- 10. sp_create_order - ouverture d'une commande au statut BROUILLON
--     (première étape du cas d'utilisation « Passer une commande »).
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_create_order $$
CREATE PROCEDURE sp_create_order (
  IN  p_id_client  INT UNSIGNED,
  IN  p_adresse    VARCHAR(255),
  OUT p_id_commande INT UNSIGNED,
  OUT p_numero      VARCHAR(20)
)
BEGIN
  DECLARE v_seq INT DEFAULT 0;

  IF NOT EXISTS (SELECT 1 FROM client WHERE id_client = p_id_client AND actif = 1) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CLIENT_INTROUVABLE_OU_BLOQUE';
  END IF;
  IF p_adresse IS NULL OR TRIM(p_adresse) = '' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ADRESSE_LIVRAISON_REQUISE';
  END IF;

  INSERT INTO commande (numero, id_client, adresse_livraison, statut)
    VALUES ('TMP', p_id_client, TRIM(p_adresse), 'BROUILLON');
  SET v_seq = LAST_INSERT_ID();
  SET p_id_commande = v_seq;
  SET p_numero = CONCAT('CMD', DATE_FORMAT(NOW(), '%Y'), '-', LPAD(v_seq, 6, '0'));
  UPDATE commande SET numero = p_numero WHERE id_commande = v_seq;
END $$

-- ----------------------------------------------------------------------
-- 11. sp_add_order_line - ajout d'une ligne dans une commande (ou d'un
```

```
--     panier persistant) : vérifie RB-05, RB-06, RB-18 ET décrémente le
--     stock. L'ordre des opérations est critique :
--        1) verrou ligne de stock (SELECT ... FOR UPDATE)
--        2) contrôle disponibilité
--        3) INSERT (le trigger trg_ligne_prix_snapshot fige le prix)
--        4) décrément (le trigger trg_ligne_decrement_stock refuse le négatif)
--        5) rafraîchissement du montant_total (RB-15)
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_add_order_line $$
CREATE PROCEDURE sp_add_order_line (
  IN  p_id_commande INT UNSIGNED,
  IN  p_id_produit  INT UNSIGNED,
  IN  p_quantite    INT,
  OUT p_id_ligne    BIGINT UNSIGNED,
  OUT p_total       DECIMAL(10,2),
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  DECLARE v_statut_cmd VARCHAR(20);
  DECLARE v_port_cmd     DECIMAL(10,2);
  DECLARE v_statut        VARCHAR(20);
  DECLARE v_prix          DECIMAL(10,2);
  DECLARE v_stock         INT;
  DECLARE v_deja          INT DEFAULT 0;
  DECLARE v_dispo         INT;

  SET p_id_ligne  = NULL;
  SET p_total     = NULL;
  SET p_code_retour = 'OK';

  -- (1) RB-05 : quantite strictement positive, controlee AVANT toute ecriture
  IF p_quantite IS NULL OR p_quantite <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB05_QUANTITE_DOIT_ETRE_SUPERIEURE_A_ZERO';
  END IF;

  -- (2) verrou sur l'en-tete de commande : une commande ne se modifie plus
  --     une fois Expediee/Livree/Annulee
  SELECT statut INTO v_statut FROM commande WHERE id_commande = p_id_commande FOR UPDATE;
  IF v_statut IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_INTROUVABLE';
  END IF;
  IF v_statut NOT IN ('BROUILLON','EN_PREPARATION') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_NON_MODIFIABLE';
  END IF;

  -- (3) RB-17 : le produit a pu disparaitre du catalogue entre l'ajout au
  --     panier et la validation -> verrou du stock + lecture du prix courant
  SELECT prix_ttc, stock INTO v_prix, v_stock
    FROM produit WHERE id_produit = p_id_produit FOR UPDATE;
  IF v_prix IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PRODUIT_SUPPRIME_DU_CATALOGUE';
  END IF;

  -- (4) RB-18 : disponibilite = stock physique - ce qui est deja reserve sur
  --     la commande (plusieurs lignes pour un meme produit sont agregees)
  SELECT IFNULL(SUM(quantite), 0) INTO v_deja
```

```
    FROM ligne_commande
   WHERE id_commande = p_id_commande AND id_produit = p_id_produit;
  SET v_dispo = v_stock - v_deja;

  IF v_dispo < p_quantite THEN
    SET p_code_retour = 'STOCK_INSUFFISANT';
  ELSE
    -- (5) une seule ecriture : trg_ligne_prix_snapshot fige le prix (RB-06),
    --     trg_ligne_decrement_stock retire les unites (RB-18)
    INSERT INTO ligne_commande (id_commande, id_produit, quantite, prix_unitaire, total_ligne)
      VALUES (p_id_commande, p_id_produit, p_quantite, v_prix, ROUND(p_quantite * v_prix, 2));
    SET p_id_ligne = LAST_INSERT_ID();

    -- (6) RB-15 : le montant de l'en-tete est une somme, jamais une saisie
    -- (6 bis) ENF-14 : tant que la commande n'est pas validee (BROUILLON), le port est
    -- recalcule a chaque ligne, sinon la regle de franchise dependrait de l'ordre des
    -- lignes. Apres validation, le port est fige (snapshot) comme le prix (RB-06).
    SELECT c.statut, c.frais_port INTO v_statut_cmd, v_port_cmd
      FROM commande c WHERE c.id_commande = p_id_commande;
    IF v_statut_cmd = 'BROUILLON' THEN
      CALL sp_compute_shipping((SELECT IFNULL(SUM(l.total_ligne), 0)
                                  FROM ligne_commande l WHERE l.id_commande = p_id_commande),
                               @v_port_new, @v_motif);
      SET v_port_cmd = IFNULL(@v_port_new, v_port_cmd);
    END IF;

    UPDATE commande c
       SET c.frais_port = v_port_cmd,
           c.montant_total = (SELECT IFNULL(SUM(l.total_ligne), 0)
                                FROM ligne_commande l WHERE l.id_commande = c.id_commande)
                           + v_port_cmd          -- RB-15 : marchandises + port
     WHERE c.id_commande = p_id_commande;
  END IF;

  -- p_total sert aussi au cas STOCK_INSUFFISANT : l'IHM affiche
  -- « X deja reserves, il en reste N » a partir de ce montant et du stock rendu.
  SELECT IFNULL(SUM(total_ligne), 0) INTO p_total
    FROM ligne_commande WHERE id_commande = p_id_commande;

  -- Note gestion des erreurs : si un trigger refuse quand meme l'ecriture
  -- (stock passe negatif a cause d'une ecriture concurrente), le SIGNAL
  -- '45000' remonte a l'appelant : le modele PHP annule la transaction
  -- (rollback). La base garde donc le dernier mot meme si le code applicatif
  -- est faux - c'est l'interet de la defense en profondeur (SEC-11).
END $$

-- ---------------------------------------------------------------------
-- 12. sp_create_order_from_basket - commande à partir d'un tableau JSON
--     [{"id_produit":1,"quantite":2}, ...] : une seule ligne d'appel depuis
--     PHP, une seule transaction, un verrou de stock par produit.
--     Démonstration du scénario nominal du cas d'utilisation UC-07.
-- ---------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_create_order_from_basket $$
CREATE PROCEDURE sp_create_order_from_basket (
  IN  p_id_client   INT UNSIGNED,
  IN  p_adresse     VARCHAR(255),
```

```
  IN  p_panier_json JSON,
  IN  p_payee       TINYINT,
  OUT p_id_commande INT UNSIGNED,
  OUT p_numero      VARCHAR(20),
  OUT p_montant     DECIMAL(10,2),
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  DECLARE v_id_produit INT UNSIGNED;
  DECLARE v_quantite   INT;
  DECLARE v_done       TINYINT DEFAULT 0;
  DECLARE v_lignes     INT DEFAULT 0;
  DECLARE v_code       VARCHAR(40);
  DECLARE cur CURSOR FOR
      SELECT j.id_produit, j.quantite FROM JSON_TABLE(p_panier_json, '$[*]'
        COLUMNS (id_produit INT UNSIGNED PATH '$.id_produit',
                 quantite   INT         PATH '$.quantite')) AS j;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

  SET p_code_retour = 'OK';

  IF p_panier_json IS NULL OR JSON_LENGTH(p_panier_json) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB19_PANIER_VIDE';
  END IF;

  START TRANSACTION;
    CALL sp_create_order(p_id_client, p_adresse, p_id_commande, p_numero);

    OPEN cur;
    la: LOOP
      FETCH cur INTO v_id_produit, v_quantite;
      IF v_done = 1 THEN LEAVE la; END IF;
      CALL sp_add_order_line(p_id_commande, v_id_produit, v_quantite, @id_ligne, @tot, v_code);
      IF v_code = 'STOCK_INSUFFISANT' THEN
        SET p_code_retour = 'STOCK_INSUFFISANT';
      END IF;
      SET v_lignes = v_lignes + 1;
    END LOOP;
    CLOSE cur;

    IF p_code_retour <> 'OK' THEN
      ROLLBACK;
      SET p_id_commande = NULL; SET p_numero = NULL; SET p_montant = NULL;
    ELSE
      UPDATE commande c
         SET c.montant_total = (SELECT IFNULL(SUM(l.total_ligne), 0)
                                  FROM ligne_commande l WHERE l.id_commande = c.id_commande)
                             + c.frais_port          -- RB-15 : marchandises + port figé
       WHERE c.id_commande = p_id_commande;
      SET v_code = NULL;
      UPDATE commande
         SET statut = IF(IFNULL(p_payee, 0) = 1, 'PAYEE', 'EN_PREPARATION')
       WHERE id_commande = p_id_commande;
      SELECT montant_total INTO p_montant FROM commande WHERE id_commande = p_id_commande;
      COMMIT;
    END IF;
```

```
END $$

-- ---------------------------------------------------------------------
-- 13. sp_confirm_order - validation du panier (RB-04 : au moins une ligne)
--     Passe BROUILLON -> EN_PREPARATION (ou PAYEE si paiement simulé).
-- ---------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_confirm_order $$
CREATE PROCEDURE sp_confirm_order (
  IN  p_id_commande INT UNSIGNED,
  IN  p_id_client   INT UNSIGNED,
  IN  p_payee       TINYINT,
  OUT p_montant     DECIMAL(10,2),
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  DECLARE v_id_client  INT UNSIGNED;
  DECLARE v_statut     VARCHAR(20);
  DECLARE v_nb         INT DEFAULT 0;
  DECLARE v_montant    DECIMAL(10,2);
  DECLARE v_manquant   INT DEFAULT 0;

  SELECT id_client, statut, montant_total INTO v_id_client, v_statut, v_montant
    FROM commande WHERE id_commande = p_id_commande FOR UPDATE;

  IF v_statut IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_INTROUVABLE';
  END IF;
  -- RB-15 / SEC-08 : contrôle d'appartenance côté serveur, la requête HTTP
  -- peut très bien porter le numéro d'une commande d'un autre client.
  IF p_id_client IS NOT NULL AND v_id_client <> p_id_client THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ACCES_NON_AUTORISE';
  END IF;
  IF v_statut <> 'BROUILLON' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_DEJA_VALIDEE';
  END IF;

  SELECT COUNT(*) INTO v_nb FROM ligne_commande WHERE id_commande = p_id_commande;
  IF v_nb = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB04_COMMANDE_DOIT_CONTENIR_AU_MOINS_UNE_LIGNE';
  END IF;

  -- RB-18 : les stocks déjà réservés par les lignes sont suffisants par
  -- construction (le décrément a eu lieu à l'ajout) ; on re-vérifie quand
  -- même qu'aucun produit n'est passé négatif entre-temps.
  SELECT COUNT(*) INTO v_manquant
    FROM ligne_commande l JOIN produit p ON p.id_produit = l.id_produit
   WHERE l.id_commande = p_id_commande AND p.stock < 0;
  IF v_manquant > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'STOCK_NEGATIF_DETECTE';
  END IF;

  -- le port est fige a la validation (snapshot, meme logique que RB-06 pour le prix) :
  -- une evolution ulterieure des tariffs de transport ne doit pas retoucher la commande
  CALL sp_compute_shipping((SELECT IFNULL(SUM(l.total_ligne), 0)
                              FROM ligne_commande l WHERE l.id_commande = p_id_commande),
                           @v_port, @v_motif);
```

```
  UPDATE commande c
     SET c.frais_port = IFNULL(@v_port, 0),
         c.montant_total = (SELECT IFNULL(SUM(l.total_ligne), 0)
                                FROM ligne_commande l WHERE l.id_commande = c.id_commande)
                           + IFNULL(@v_port, 0),
         c.statut = IF(IFNULL(p_payee, 0) = 1, 'PAYEE', 'EN_PREPARATION')
   WHERE c.id_commande = p_id_commande;

  SELECT montant_total INTO p_montant FROM commande WHERE id_commande = p_id_commande;
  SET p_code_retour = 'OK';
END $$

-- ----------------------------------------------------------------------
-- 14. sp_update_order_status - changement de statut (EF-ADM-06, RB-11)
--       La matrice des transitions est contrôlée ici ET par le trigger
--       trg_commande_transition_statut (défense en profondeur).
-- ----------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_update_order_status $$
CREATE PROCEDURE sp_update_order_status (
  IN  p_id_commande   INT UNSIGNED,
  IN  p_nouveau_statut VARCHAR(20),
  IN  p_commentaire   VARCHAR(500),
  IN  p_id_auteur     INT UNSIGNED,
  IN  p_role_auteur   VARCHAR(10),
  OUT p_code_retour   VARCHAR(40)
)
BEGIN
  DECLARE v_statut VARCHAR(20);

  IF p_id_commande IS NULL OR p_nouveau_statut IS NULL
     OR p_nouveau_statut NOT IN
('BROUILLON','EN_PREPARATION','PAYEE','EXPEDIEE','LIVREE','ANNULEE') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'STATUT_NON_AUTORISE';
  END IF;

  SELECT statut INTO v_statut FROM commande WHERE id_commande = p_id_commande FOR UPDATE;
  IF v_statut IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_INTROUVABLE';
  END IF;
  -- règle de métier : le client annule tant que ce n'est pas expédié ;
  -- l'administrateur peut en plus rejeter une commande en préparation.
  IF p_role_auteur = 'CLIENT' AND v_statut NOT IN ('BROUILLON','EN_PREPARATION','PAYEE') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ANNULATION_TROP_TARDIVE';
  END IF;
  IF v_statut = p_nouveau_statut THEN
    SET p_code_retour = 'STATUT_DEJA_A_JOUR';
  ELSE
    UPDATE commande
       SET statut = p_nouveau_statut,
           commentaire = IFNULL(p_commentaire, commentaire)
     WHERE id_commande = p_id_commande;
    -- PAS d'INSERT dans order_status_history : trg_history_statut (AFTER UPDATE)
    -- s'en charge. Laisser la procédure écrire aussi produirait un doublon
    -- (et l'application pourrait « oublier » de tracer : le trigger, lui, n'oublie jamais).
    SET p_code_retour = 'OK';
```

```
  END IF;
END $$

-- ---------------------------------------------------------------------
-- 15. sp_cancel_order - annulation par le client (EF-CLI-05, RB-18) :
--     restauration ligne à ligne du stock PUIS changement de statut,
--     le tout dans une transaction unique (aucune ligne « orpheline »).
-- ---------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_cancel_order $$
CREATE PROCEDURE sp_cancel_order (
  IN  p_id_commande INT UNSIGNED,
  IN  p_id_client   INT UNSIGNED,
  OUT p_code_retour VARCHAR(40)
)
BEGIN
  DECLARE v_id_produit INT UNSIGNED;
  DECLARE v_quantite   INT;
  DECLARE v_done       TINYINT DEFAULT 0;
  DECLARE v_statut     VARCHAR(20);
  DECLARE v_id_client  INT UNSIGNED;
  DECLARE cur CURSOR FOR
      SELECT id_produit, quantite FROM ligne_commande WHERE id_commande = p_id_commande;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

  SELECT id_client, statut INTO v_id_client, v_statut
    FROM commande WHERE id_commande = p_id_commande FOR UPDATE;
  IF v_statut IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_INTROUVABLE';
  END IF;
  IF p_id_client IS NOT NULL AND v_id_client <> p_id_client THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ACCES_NON_AUTORISE';
  END IF;
  IF v_statut NOT IN ('BROUILLON','EN_PREPARATION','PAYEE') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'COMMANDE_NON_ANNULABLE';
  END IF;

  START TRANSACTION;
    -- La restitution du stock n'est PAS faite ici : DELETE sur ligne_commande
    -- declenche trg_ligne_restore_stock, qui rend les unites ligne par ligne.
    -- Le cursor sert uniquement a memoriser les quantites restituees pour le
    -- message renvoye a l'IHM (et a prouver qu'on sait lire OLD/NEW... depuis
    -- une procedure). Le trigger reste la source de verite (RB-18).
    OPEN cur;
    SET @qte_restauree = 0;
    lp: LOOP
      FETCH cur INTO v_id_produit, v_quantite;
      IF v_done = 1 THEN LEAVE lp; END IF;
      SET @qte_restauree = @qte_restauree + v_quantite;
    END LOOP;
    CLOSE cur;
    DELETE FROM ligne_commande WHERE id_commande = p_id_commande;
    -- Le port est annule avec les lignes : une commande annulee n'a plus rien a
    -- facturer, et la formule RB-15 (montant = lignes + port) doit rester vraie a
    -- chaque ecriture, y compris pour ANNULEE (le trigger la verifie sans condition).
    UPDATE commande SET statut = 'ANNULEE',
                        frais_port = 0.00,
```

```
                        montant_total = 0.00,
                        commentaire = IFNULL(commentaire, 'Annulation demandée par le client')
      WHERE id_commande = p_id_commande;   -- trg_history_statut trace EN_PREPARATION -> ANNULEE
  COMMIT;
  SET p_code_retour = 'OK';
END $$

-- ---------------------------------------------------------------------
-- 16. sp_revenue_report - tableau de bord Back-office (EF-ADM-07)
-- ---------------------------------------------------------------------
DROP PROCEDURE IF EXISTS sp_revenue_report $$
CREATE PROCEDURE sp_revenue_report (IN p_deb DATETIME, IN p_fin DATETIME)
BEGIN
  SELECT c.statut,
         COUNT(*)                                    AS nb_commandes,
         IFNULL(SUM(c.montant_total), 0)              AS montant_ht_eq_ttc,
         IFNULL(ROUND(AVG(c.montant_total), 2), 0)    AS montant_moyen
    FROM commande c
   WHERE (p_deb IS NULL OR c.date_commande >= p_deb)
     AND (p_fin IS NULL OR c.date_commande <  p_fin)
   GROUP BY c.statut
   ORDER BY montant_ht_eq_ttc DESC;

  SELECT p.reference, p.nom, SUM(l.quantite) AS unites, SUM(l.total_ligne) AS ca
    FROM ligne_commande l
    JOIN commande c ON c.id_commande = l.id_commande AND c.statut <> 'ANNULEE'
    JOIN produit  p ON p.id_produit  = l.id_produit
   WHERE (p_deb IS NULL OR c.date_commande >= p_deb)
     AND (p_fin IS NULL OR c.date_commande <  p_fin)
   GROUP BY p.reference, p.nom
   ORDER BY ca DESC
   LIMIT 5;
END $$
```

C. Déclencheurs - `sql/03_minishop_triggers.sql`

(293 lignes, copié verbatim depuis le dépôt.)

```
-- ======================================================================
-- MiniShop - 03 : déclencheurs (16 triggers, minimum demandé : 5)
-- ----------------------------------------------------------------------
-- Principes :
--   * Un trigger ne peut pas lire la table qu'il est en train de muter
--     (limitation MySQL/MariaDB) : chaque règle est donc écrite « du bon
--     côté » (déclencheur sur la table fille pour la table parente, et
--     inversement).
--   * Les triggers sont la DERNIÈRE barrière : même un UPDATE écrit à la
--     main depuis mysql, phpMyAdmin ou un contrôleur non MVC ne peut pas
--     violer RB-02 / RB-03 / RB-05 / RB-06 / RB-09 / RB-11 / RB-14.
--   * ORDER_STATUS_HISTORY n'est alimentée QUE par trg_history_statut :
--     l'application ne peut pas « oublier » de tracer (audit fiable).
--   * ORDRE DE CRÉATION : MySQL exécute les triggers d'un même évènement
--     dans l'ordre de leur création. trg_ligne_immutable est donc créé
--     AVANT trg_ligne_prix_snapshot, sans quoi une modification du prix
```

```
--     d'une ligne serait silencieusement « réparée » par le snapshot au
--     lieu d'être rejetée (cf. test T-11).
--
-- Chargement :  mysql -u root -p minishop < sql/03_minishop_triggers.sql
--               (ou scripts/load_db.sh qui encapsule le DELIMITER $$)
-- ======================================================================

-- ======================================================================
-- BLOC A - STOCK (RB-03, RB-18) : le stock ne peut jamais devenir négatif
-- ======================================================================

-- TRG-A1 - contrôle de disponibilité avant l'insertion d'une ligne
DROP TRIGGER IF EXISTS trg_ligne_controle_insert $$
CREATE TRIGGER trg_ligne_controle_insert
BEFORE INSERT ON ligne_commande
FOR EACH ROW
BEGIN
  DECLARE v_prix DECIMAL(10,2);
  DECLARE v_stock INT;

  SELECT prix_ttc, stock INTO v_prix, v_stock
    FROM produit WHERE id_produit = NEW.id_produit;

  IF v_prix IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PRODUIT_INTROUVABLE';    -- RB-17
  END IF;
  IF v_stock < NEW.quantite THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'STOCK_INSUFFISANT';      -- RB-18
  END IF;
END $$

-- TRG-A2 - décrément du stock physique après insertion de la ligne
DROP TRIGGER IF EXISTS trg_ligne_decrement_stock $$
CREATE TRIGGER trg_ligne_decrement_stock
AFTER INSERT ON ligne_commande
FOR EACH ROW
BEGIN
  UPDATE produit
     SET stock = GREATEST(stock - NEW.quantite, 0)
   WHERE id_produit = NEW.id_produit;
END $$

-- TRG-A3 - restauration du stock quand une ligne disparaît
--          (annulation de commande, retrait du panier, purge d'un brouillon)
DROP TRIGGER IF EXISTS trg_ligne_restore_stock $$
CREATE TRIGGER trg_ligne_restore_stock
AFTER DELETE ON ligne_commande
FOR EACH ROW
BEGIN
  UPDATE produit
     SET stock = stock + OLD.quantite
   WHERE id_produit = OLD.id_produit;
END $$

-- ======================================================================
-- BLOC B - PRIX, IMMUTABILITÉ (RB-02, RB-06, RB-09, RB-14, RB-16)
```

```
-- ========================================================================

-- TRG-B1 - une ligne de commande est définitive : produit, quantité et prix
--          ne se réécrivent plus (RB-09 + RB-06).
DROP TRIGGER IF EXISTS trg_ligne_immutable $$
CREATE TRIGGER trg_ligne_immutable
BEFORE UPDATE ON ligne_commande
FOR EACH ROW
BEGIN
  IF NEW.quantite <> OLD.quantite
     OR NEW.id_produit <> OLD.id_produit
     OR NEW.prix_unitaire <> OLD.prix_unitaire THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB06_PRIX_ET_QUANTITE_NON_MODIFIABLE';
  END IF;
END $$

-- TRG-B2 - snapshot du prix : le prix enregistré dans LIGNE_COMMANDE est
--          TOUJOURS le prix TTC du produit à l'instant de l'achat, même si le
--          navigateur (ou un code mal écrit) envoie un autre prix (RB-06).
--          NB : NEW.<colonne> n'est pas une cible valide pour SELECT ... INTO
--          dans un trigger, d'où la variable locale.
DROP TRIGGER IF EXISTS trg_ligne_prix_snapshot $$
CREATE TRIGGER trg_ligne_prix_snapshot
BEFORE INSERT ON ligne_commande
FOR EACH ROW
BEGIN
  DECLARE v_prix DECIMAL(10,2);

  SELECT p.prix_ttc INTO v_prix
    FROM produit p WHERE p.id_produit = NEW.id_produit;
  IF v_prix IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PRODUIT_INTROUVABLE';
  END IF;
  SET NEW.prix_unitaire = v_prix;
  SET NEW.total_ligne   = ROUND(NEW.quantite * v_prix, 2);
END $$

-- TRG-B3 - prix TTC calculé par la base, jamais transmis par le navigateur
DROP TRIGGER IF EXISTS trg_produit_ttc $$
CREATE TRIGGER trg_produit_ttc
BEFORE INSERT ON produit
FOR EACH ROW
BEGIN
  SET NEW.prix_ttc = ROUND(NEW.prix_ht * (1 + NEW.tva / 100), 2);
  -- etat de reference pour la campagne de tests : la quantite du seed est memorisee
  -- a l'insertion, ainsi la fixture peut toujours repartir du meme stock sans
  -- dependre de l'ordre ni du nombre de passages des tests.
  IF NEW.stock_initial IS NULL OR NEW.stock_initial < 0 THEN
    SET NEW.stock_initial = IFNULL(NEW.stock, 0);
  END IF;
END $$

-- TRG-B4 - idem en modification : toute évolution du prix HT ou du taux
--          recalcule le TTC, sans jamais toucher aux lignes déjà enregistrées
DROP TRIGGER IF EXISTS trg_produit_ttc_update $$
CREATE TRIGGER trg_produit_ttc_update
```

```
BEFORE UPDATE ON produit
FOR EACH ROW
  SET NEW.prix_ttc = ROUND(NEW.prix_ht * (1 + NEW.tva / 100), 2);

-- TRG-B5 - garde-fou sur le stock saisi en Back-office (RB-03). Le CHECK
--          ck_produit_stock bloque déjà la valeur ; ce trigger fournit un
--          message explicite à l'IHM.
DROP TRIGGER IF EXISTS trg_produit_regles $$
CREATE TRIGGER trg_produit_regles
BEFORE UPDATE ON produit
FOR EACH ROW
BEGIN
  IF NEW.stock < 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF';
  END IF;
END $$

-- TRG-B6 - RB-14 : un produit référencé par une commande ne peut pas être
--          supprimé (le snapshot RB-06 doit rester explicable dans 5 ans).
DROP TRIGGER IF EXISTS trg_produit_delete $$
CREATE TRIGGER trg_produit_delete
BEFORE DELETE ON produit
FOR EACH ROW
BEGIN
  IF EXISTS (SELECT 1 FROM ligne_commande WHERE id_produit = OLD.id_produit) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PRODUIT_REFERENCE_INTERDIT_DE_SUPPRIMER';
  END IF;
END $$

-- ====================================================================
-- BLOC C - CATÉGORIES (RB-07, RB-08, RB-14)
-- ====================================================================

-- TRG-C1 - normalisation du slug (permet des URLs lisibles et stables)
DROP TRIGGER IF EXISTS trg_categorie_slug $$
CREATE TRIGGER trg_categorie_slug
BEFORE INSERT ON categorie
FOR EACH ROW
BEGIN
  IF NEW.slug IS NULL OR TRIM(NEW.slug) = '' THEN
    SET NEW.slug = LOWER(REPLACE(NEW.nom, ' ', '-'));
  END IF;
END $$

-- TRG-C2 - on ne supprime pas une catégorie qui porte encore des produits
DROP TRIGGER IF EXISTS trg_categorie_delete $$
CREATE TRIGGER trg_categorie_delete
BEFORE DELETE ON categorie
FOR EACH ROW
BEGIN
  IF EXISTS (SELECT 1 FROM produit WHERE id_categorie = OLD.id_categorie) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CATEGORIE_NON_VIDE';
  END IF;
END $$

-- ====================================================================
```

```
-- BLOC D - COMMANDES, STATUTS, AUDIT (RB-04, RB-10, RB-11, RB-15)
-- ======================================================================

-- TRG-D1 - matrice des transitions autorisées + montants non saisissables
--          + rattachement à un client réel (RB-10, RB-11, RB-15)
DROP TRIGGER IF EXISTS trg_commande_transition_statut $$
CREATE TRIGGER trg_commande_transition_statut
BEFORE UPDATE ON commande
FOR EACH ROW
BEGIN
  IF NOT EXISTS (SELECT 1 FROM client WHERE id_client = NEW.id_client) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB10_COMMANDE_SANS_CLIENT';
  END IF;

  -- RB-15 / ENF-14 : une fois la commande validee, `montant_total` DOIT rester egal a
  -- (somme des lignes + port) : le controle n'est plus conditionne a un changement du
  -- montant, sinon un UPDATE qui ne touche QUE `frais_port` laisserait la facture
  -- incoherente sans jamais etre refuse. Les brouillons restent tolerants (RB-04
  -- n'exige une ligne qu'a la validation).
  IF (NEW.statut = 'BROUILLON'
        AND NEW.montant_total <> OLD.montant_total
        AND NEW.montant_total <> (SELECT IFNULL(SUM(total_ligne), 0)
                                    FROM ligne_commande
                                   WHERE id_commande = NEW.id_commande) + NEW.frais_port)
     OR (NEW.statut <> 'BROUILLON'
        AND NEW.montant_total <> (SELECT IFNULL(SUM(total_ligne), 0)
                                    FROM ligne_commande
                                   WHERE id_commande = NEW.id_commande) + NEW.frais_port) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB15_MONTANT_CALCULE_INTERDIT';
  END IF;

  IF NEW.statut <> OLD.statut THEN
    IF (OLD.statut = 'BROUILLON'      AND NEW.statut NOT IN ('EN_PREPARATION','PAYEE','ANNULEE'))
    OR (OLD.statut = 'EN_PREPARATION' AND NEW.statut NOT IN ('PAYEE','EXPEDIEE','ANNULEE'))
    OR (OLD.statut = 'PAYEE'          AND NEW.statut NOT IN ('EXPEDIEE','ANNULEE'))
    OR (OLD.statut = 'EXPEDIEE'       AND NEW.statut NOT IN ('LIVREE','ANNULEE'))
    OR (OLD.statut IN ('LIVREE','ANNULEE')) THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB11_TRANSITION_STATUT_INTERDITE';
    END IF;
  END IF;
END $$

-- TRG-D2 - RB-04 / RB-10 / RB-15 à la création : client réel, montant à 0,
--          statut initial forcé à BROUILLON
DROP TRIGGER IF EXISTS trg_commande_controle_insert $$
CREATE TRIGGER trg_commande_controle_insert
BEFORE INSERT ON commande
FOR EACH ROW
BEGIN
  IF NOT EXISTS (SELECT 1 FROM client WHERE id_client = NEW.id_client) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB10_COMMANDE_SANS_CLIENT';
  END IF;
  IF NEW.montant_total <> 0 THEN
    SET NEW.montant_total = 0;   -- RB-15 : le montant se calcule, il ne se saisit pas
  END IF;
  IF NEW.frais_port IS NULL OR NEW.frais_port < 0 THEN
```

```
    SET NEW.frais_port = 0;      -- RB-15 : le port est figure par sp_compute_shipping, jamais
saisi a la main
  END IF;
  IF NEW.statut IS NULL THEN
    SET NEW.statut = 'BROUILLON';
  END IF;
END $$

-- TRG-D3 - on ne purge pas l'historique : suppression interdite dès que la
--          commande a été validée ; une commande ne disparaît que vidée de
--          ses lignes (cohérence avec RB-04)
DROP TRIGGER IF EXISTS trg_commande_delete $$
CREATE TRIGGER trg_commande_delete
BEFORE DELETE ON commande
FOR EACH ROW
BEGIN
  IF OLD.statut NOT IN ('BROUILLON','EN_PREPARATION') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB09_COMMANDE_VALIDATEE_INTERDITE_DE_SUPPRIMER';
  END IF;
  IF EXISTS (SELECT 1 FROM ligne_commande WHERE id_commande = OLD.id_commande) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RB04_COMMANDE_DOIT_CONTENIR_AU_MOINS_UNE_LIGNE';
  END IF;
END $$

-- TRG-D4 - HISTORIQUE AUTOMATIQUE (exigence « Trigger 2 - Historique ») :
--          chaque changement de statut est tracé, que la modification vienne
--          d'une procédure stockée, d'un import de reprise ou d'un UPDATE
--          manuel en session.
DROP TRIGGER IF EXISTS trg_history_statut $$
CREATE TRIGGER trg_history_statut
AFTER UPDATE ON commande
FOR EACH ROW
BEGIN
  IF NEW.statut <> OLD.statut THEN
    INSERT INTO order_status_history (order_id, old_status, new_status, changed_by_role,
commentaire)
      VALUES (NEW.id_commande, OLD.statut, NEW.statut, 'SYSTEME',
              CONCAT('Tracé par trg_history_statut le ', DATE_FORMAT(NOW(), '%Y-%m-%d %H:%i')));
  END IF;
END $$

-- TRG-D5 - le statut initial est également tracé (première ligne d'historique)
DROP TRIGGER IF EXISTS trg_history_creation $$
CREATE TRIGGER trg_history_creation
AFTER INSERT ON commande
FOR EACH ROW
  INSERT INTO order_status_history (order_id, old_status, new_status, changed_by_role)
    VALUES (NEW.id_commande, NULL, NEW.statut, 'SYSTEME');

-- ======================================================================
-- Vérification : la liste des triggers installés
--   SELECT TRIGGER_NAME, EVENT_OBJECT_TABLE, ACTION_TIMING, EVENT_MANIPULATION
--     FROM information_schema.TRIGGERS WHERE TRIGGER_SCHEMA = DATABASE()
--    ORDER BY EVENT_OBJECT_TABLE, ACTION_TIMING;
-- ======================================================================
```

D. Commandes de démonstration via `CALL` - `sql/04_minishop_demo.sql`

(46 lignes, copié verbatim depuis le dépôt.)

```
-- ======================================================================
-- MiniShop - 04 : jeu de données de démonstration (commandes)
-- ----------------------------------------------------------------------
-- Les commandes sont créées EN PASSANT PAR LES PROCEDURES STOCKÉES :
-- c'est la seule façon d'exécuter réellement les triggers (snapshot du
-- prix, décrément de stock, historique des statuts).
-- Rejeu possible : le script 01 reconstruit la base de zéro.
-- 4 commandes : une validee sous franchise de port, un brouillon vide, une expediee,
-- et une petite commande qui PAYE les frais de port standard (demonstration ENF-14).
-- ======================================================================

-- Nom de la base : passe au client (`mariadb minishop < fichier`) par
-- scripts/load_db.sh ; en mode interactif, taper `USE minishop;` d'abord.

-- Client 1 : commande complète (2 x Casque Aura + 1 x Cable USB-C) validée
CALL sp_create_order(1, '12 rue de France, 06000 Nice', @id_c1, @num_c1);
CALL sp_add_order_line(@id_c1, 5, 2, @l, @t, @r);
CALL sp_add_order_line(@id_c1, 9, 1, @l, @t, @r);
-- validation par la procedure : c'est elle qui FIGE les frais de port et solde le
-- montant (RB-15, ENF-14). Marchandises = 324.60 EUR >= franchise de 80.00 EUR :
-- port offert (frais_port = 0.00) et montant_total = 324.60 EUR.
CALL sp_confirm_order(@id_c1, 1, 0, @mnt_c1, @r_c1);

-- Client 2 : panier en préparation (brouillon) - 1 x Batterie externe (stock 0)
CALL sp_create_order(2, '5 avenue Jean Médecin, 06000 Nice', @id_c2, @num_c2);

-- Client 3 : commande payée puis expédiée (statuts posés par UPDATE direct : le trigger
-- trg_commande_transition_statut contrôle la matrice RB-11 exactement comme la procédure)
CALL sp_create_order(3, '8 rue Papin, 06300 Nice', @id_c3, @num_c3);
CALL sp_add_order_line(@id_c3, 3, 1, @l, @t, @r);
CALL sp_add_order_line(@id_c3, 4, 2, @l, @t, @r);
UPDATE commande SET statut = 'EN_PREPARATION' WHERE id_commande = @id_c3;
UPDATE commande SET statut = 'PAYEE'          WHERE id_commande = @id_c3;
UPDATE commande SET statut = 'EXPEDIEE'       WHERE id_commande = @id_c3;

-- Client 1 (bis) : petit panier SOUS la franchise, pour que les frais de port soient
-- visibles dans la demonstration et dans le tableau de bord (4,90 EUR de livraison
-- standard ajoutes au montant) ; C1 et C3 beneficient en revanche de la franchise.
CALL sp_create_order(1, '12 rue de France, 06000 Nice', @id_c4, @num_c4);
CALL sp_add_order_line(@id_c4, 8, 1, @l, @t, @r);    -- 1 x Ecran 27" QHD : 58.80 EUR TTC
CALL sp_confirm_order(@id_c4, 1, 0, @mnt_c4, @r_c4); -- 58.80 < 80.00 -> port 4.90, total 63.70

SELECT c.numero, cl.email, c.statut, c.montant_total, c.frais_port,
       (SELECT COUNT(*) FROM ligne_commande l WHERE l.id_commande = c.id_commande) AS lignes
  FROM commande c JOIN client cl ON cl.id_client = c.id_client
 ORDER BY c.id_commande;
```

E. Jeu de données des tests - `tests/sql/fixture.sql`

(105 lignes, copié verbatim depuis le dépôt.)

```
-- ========================================================================
-- fixture jouee par le harnais AVANT chaque test : etat deterministe.
-- Les lignes de commande des tests sont purgees VIA UNE TABLE
-- TEMPORAIRE : un `DELETE ... WHERE id_produit IN (SELECT ... produit)`
-- echouerait (la table PRODUIT serait lue par la requete declenchante et
-- modifiee par trg_ligne_restore_stock).
-- ========================================================================
-- Etat de reference des stocks : la campagne rejoue la fixture AVANT CHAQUE test, donc
-- un stock diminue par un test precedent doit etre restitue ; sinon les tests finissent
-- par echouer pour une raison sans rapport avec la regle testee (stock epuise par
-- l'accumulation des reservations). `stock_initial` porte la quantite du seed, renseignee
-- a l'insertion par trg_produit_ttc.
UPDATE produit SET stock_initial = stock WHERE stock_initial < 0;
UPDATE produit SET stock = stock_initial;

INSERT IGNORE INTO categorie (nom, slug, description)
  VALUES ('Cat test','cat-test-t01','categorie de test pour RB-14 / RB-18');

INSERT IGNORE INTO produit (reference, nom, slug, description, prix_ht, prix_ttc, stock,
seuil_alerte, id_categorie, visible)
  VALUES ('TST-900','Produit test 900','tst-900','Stock 5, prix HT 10.00 / TTC 12.00 (attendu par
les tests).',
          10.00, 12.00, 5, 2, (SELECT id_categorie FROM categorie WHERE slug='cat-test-t01'), 1),
         ('TST-901','Produit test 901','tst-901','Stock 0 : produit en rupture.',
          20.00, 24.00, 0, 1, (SELECT id_categorie FROM categorie WHERE slug='cat-test-t01'), 1),
         ('TST-902','Produit test 902','tst-902','Prix HT 60.00 / TTC 72.00 : sert aux tests de
frais de port (T-28).',
          60.00, 72.00, 50, 5, (SELECT id_categorie FROM categorie WHERE slug='cat-test-t01'), 1);

CREATE TEMPORARY TABLE IF NOT EXISTS tmp_lignes_test (id_ligne BIGINT UNSIGNED PRIMARY KEY);
CREATE TEMPORARY TABLE IF NOT EXISTS tmp_cmd_test    (id_commande INT UNSIGNED PRIMARY KEY);
TRUNCATE tmp_lignes_test; TRUNCATE tmp_cmd_test;

INSERT IGNORE INTO tmp_cmd_test
  SELECT id_commande FROM commande WHERE numero LIKE 'TMP' OR numero LIKE 'CMD-TST%';
INSERT IGNORE INTO tmp_lignes_test
  SELECT l.id_ligne FROM ligne_commande l
    JOIN produit p ON p.id_produit = l.id_produit AND p.reference LIKE 'TST-%'
   UNION
  SELECT l.id_ligne FROM ligne_commande l JOIN tmp_cmd_test t ON t.id_commande = l.id_commande;

DELETE FROM ligne_commande WHERE id_ligne IN (SELECT id_ligne FROM tmp_lignes_test);
DELETE FROM commande        WHERE id_commande IN (SELECT id_commande FROM tmp_cmd_test);
TRUNCATE tmp_lignes_test; TRUNCATE tmp_cmd_test;

-- Purge des brouillons sans ligne laisses par les tests precedents. Un brouillon
-- vide conserve son montant_total tant qu'on ne le corrige pas ; le trigger RB-15
-- refuserait alors la normalisation ci-dessous et l'erreur se propagerait a TOUS les
-- tests suivants (effet de bord en chaine). Un brouillon sans ligne n'a aucune valeur
-- metier (RB-04 n'est exige qu'a la validation) : on le supprime proprement.
INSERT IGNORE INTO tmp_cmd_test
  SELECT c.id_commande FROM commande c
   WHERE c.statut = 'BROUILLON'
     AND NOT EXISTS (SELECT 1 FROM ligne_commande l WHERE l.id_commande = c.id_commande)
```

```
     AND (c.montant_total <> 0 OR c.frais_port <> 0);
DELETE FROM commande WHERE id_commande IN (SELECT id_commande FROM tmp_cmd_test);
TRUNCATE tmp_cmd_test;

-- Dernier filet : un brouillon vide et au solde nul est un residu de test (le seed n'en
-- produit qu'un seul, inoffensif) ; le supprimer garde la base dans un etat ou RB-15 ne
-- peut plus refuser la normalisation ci-dessous.
INSERT IGNORE INTO tmp_cmd_test
  SELECT c.id_commande FROM commande c
   WHERE c.statut = 'BROUILLON' AND c.montant_total = 0 AND c.frais_port = 0
     AND NOT EXISTS (SELECT 1 FROM ligne_commande l WHERE l.id_commande = c.id_commande);
DELETE FROM commande WHERE id_commande IN (SELECT id_commande FROM tmp_cmd_test);
TRUNCATE tmp_cmd_test;

UPDATE produit SET prix_ht = 10.00, stock = 5, visible = 1, seuil_alerte = 2
 WHERE reference = 'TST-900';
UPDATE produit SET prix_ht = 20.00, stock = 0, visible = 1 WHERE reference = 'TST-901';
UPDATE produit SET prix_ht = 60.00, stock = 50, visible = 1 WHERE reference = 'TST-902';

-- Un brouillon sans ligne avec un port figure est un residu de test : on le retire
-- avant de normaliser (le trigger interdit de « reparer » son montant sans corriger
-- le port dans la meme instruction, et un residu bloquerait tous les tests suivants).
INSERT IGNORE INTO tmp_cmd_test
  SELECT c.id_commande FROM commande c
   WHERE c.statut = 'BROUILLON' AND c.frais_port > 0
     AND NOT EXISTS (SELECT 1 FROM ligne_commande l WHERE l.id_commande = c.id_commande);
DELETE FROM commande WHERE id_commande IN (SELECT id_commande FROM tmp_cmd_test);
TRUNCATE tmp_cmd_test;

-- Normalisation des montants des commandes rescapées : le retrait des lignes de test
-- est fait par DELETE direct (la sous-requête sur produit est interdite à cause du
-- trigger qui écrit sur produit), donc `trg_ligne_decrement_stock` et le recalcul du
-- total n'ont pas pu se déclencher. On remet montant_total = somme des lignes pour
-- qu'aucun test ne bute sur une incohérence créée par la fixture elle-même.
-- On remet FRAIS_PORT ET montant_total dans la meme instruction : le trigger
-- trg_commande_transition_statut verifie « montant = somme des lignes + port » a
-- partir de NEW.frais_port ; changer le port tout seul violerait done RB-15 et
-- bloquerait la fixture (et, par contagion, les tests suivants).
UPDATE commande c
   SET c.frais_port = 0.00,
       c.montant_total = (SELECT COALESCE(SUM(l.total_ligne), 0)
                            FROM ligne_commande l WHERE l.id_commande = c.id_commande)
 WHERE c.statut = 'BROUILLON'
   AND c.montant_total <> (SELECT COALESCE(SUM(l.total_ligne), 0)
                             FROM ligne_commande l WHERE l.id_commande = c.id_commande)
       + c.frais_port;
UPDATE commande c
   SET c.montant_total = (SELECT COALESCE(SUM(l.total_ligne), 0)
                            FROM ligne_commande l WHERE l.id_commande = c.id_commande)
                       + c.frais_port
 WHERE c.statut <> 'BROUILLON'
   AND c.montant_total <> (SELECT COALESCE(SUM(l.total_ligne), 0)
                             FROM ligne_commande l WHERE l.id_commande = c.id_commande)
       + c.frais_port;
```

F. Manifeste des tests SQL - `tests/sql/manifest.txt`

(38 lignes, copié verbatim depuis le dépôt.)

```
# ======================================================================
# MiniShop - manifeste de la suite de tests SQL
# Colonnes separees par "|":
#   id|regle|description du test|resultat attendu (ERREUR:<motif> ou OK)|fichier SQL
# Le harnais tests/sql/run_tests.sh cree une base de test, applique le
# fichier, puis compare le resultat reel a l'attendu.
#  "ERREUR:<motif>" : le fichier DOIT echouer et mysql doit afficher <motif>
#  "OK"             : le fichier DOIT reussir (et afficher les PASS attendus)
# ======================================================================
T-01|RB-02|prix HT = 0 refuse par le CHECK
ck_produit_prix|ERREUR:ck_produit_prix|t01_prix_zero.sql
T-02|RB-03|stock negatif refuse (CHECK) en ecriture
directe|ERREUR:ck_produit_stock|t02_stock_negatif.sql
T-03|RB-03|stock negatif refuse par
trg_produit_regles|ERREUR:RB03_STOCK_NE_PEUT_PAS_ETRE_NEGATIF|t03_trigger_stock.sql
T-04|RB-01|email client duplique (contrainte
unique)|ERREUR:uk_client_email|t04_email_duplicate.sql
T-05|RB-12|sp_create_account refuse un mot de passe non
hashe|ERREUR:HASH_MOT_DE_PASSE_INVALIDE|t05_hash_obligatoire.sql
T-06|RB-05|quantite = 0 refusee par
sp_add_order_line|ERREUR:RB05_QUANTITE_DOIT_ETRE_SUPERIEURE_A_ZERO|t06_quantite_zero.sql
T-07|RB-05|quantite = 0 refusee par le CHECK
ck_ligne_quantite|ERREUR:ck_ligne_quantite|t07_check_quantite.sql
T-08|RB-18|stock insuffisant : le trigger trg_ligne_controle_insert
bloque|ERREUR:STOCK_INSUFFISANT|t08_stock_insuffisant.sql
T-09|RB-18|stock insuffisant : code de retour de la procedure|OK|t09_stock_insuffisant_sp.sql
T-10|RB-15|montant_total non saisissable
(trigger)|ERREUR:RB15_MONTANT_CALCULE_INTERDIT|t10_montant_saisi.sql
T-11|RB-06|prix unitaire non modifiable
(immutabilite)|ERREUR:RB06_PRIX_ET_QUANTITE_NON_MODIFIABLE|t11_prix_modifie.sql
T-12|RB-11|transition EN_PREPARATION -> BROUILLON
interdite|ERREUR:RB11_TRANSITION_STATUT_INTERDITE|t12_transition.sql
T-13|RB-11|transition EXPEDIEE -> BROUILLON
interdite|ERREUR:RB11_TRANSITION_STATUT_INTERDITE|t13_transition2.sql
T-14|RB-14|suppression d'une categorie non
vide|ERREUR:CATEGORIE_NON_VIDE|t14_categorie_occupee.sql
T-15|RB-14|suppression d'un produit deja
commande|ERREUR:PRODUIT_REFERENCE_INTERDIT_DE_SUPPRIMER|t15_produit_commande.sql
T-16|RB-04|validation d'une commande sans
ligne|ERREUR:RB04_COMMANDE_DOIT_CONTENIR_AU_MOINS_UNE_LIGNE|t16_commande_vide.sql
T-17|RB-10|commande rattachee a un client inexistant (trigger +
FK)|ERREUR:RB10_COMMANDE_SANS_CLIENT|t17_client_inexistant.sql
T-18|SEC-08|sp_confirm_order refuse l'acces a la commande d'un autre
client|ERREUR:ACCES_NON_AUTORISE|t18_a_dere.sql
T-19|RB-03|sp_adjust_stock refuse le passage en negatif (OUT)|OK|t19_adjust_negatif.sql
T-20|RB-09|produit deja commande : bascule visible=0 (pas de
suppression)|OK|t20_suppression_logique.sql
T-21|RB-18|annulation : stock restaure ligne par ligne|OK|t21_annulation.sql
T-22|RB-11|historique des statuts ecrit automatiquement par le trigger|OK|t22_historique.sql
T-23|RB-06|le prix de la ligne survit a une hausse tarifaire ulterieure|OK|t23_snapshot.sql
T-24|RB-04|suppression d'une commande BROUILLON vide autorisee|OK|t24_purge.sql
```

```
T-25|EF-VIS-02|recherche + filtres + produits masques exclus|OK|t25_recherche.sql
T-26|EF-ADM-05|alerte stock (RUPTURE / TRES_BAS) calculee par la procedure|OK|t26_alerte_stock.sql
T-27|EF-VIS-01|les 3 vues (catalogue, etat de stock, commandes client) existent et sont coherentes
 avec les tables|OK|t27_vues.sql
T-28|ENF-14|frais de port calcules avant validation, figurés a la validation, non
saisissables|ERREUR:RB15_MONTANT_CALCULE_INTERDIT|t28_port.sql
T-29|ENF-16|erreur en cours de commande : rollback integral, aucune donnee
partielle|OK|t29_rollback_commande.sql
```

G. Protocole de mesure des performances - `tests/perf/mesurer.sh`

(19 lignes, copié verbatim depuis le dépôt.)

```
#!/usr/bin/env bash
# MiniShop - mesure du temps de reponse d'une URL (ENF-01 : p95 < 500 ms).
#   URL_BASE=http://127.0.0.1:8000 N=200 ./tests/perf/mesurer.sh '/catalogue?cat=1'
set -uo pipefail
base="${URL_BASE:-http://127.0.0.1:8000}"; url="${1:-/}"; n="${N:-200}"; c="${C:-4}"
command -v ab >/dev/null 2>&1 && { ab -n "$n" -c "$c" -k "$base$url" | grep -E 'Requests per
second|Time per request|Failed requests'; exit $?; }
ttmp="$(mktemp)"; ok=0; fail=0
for i in $(seq 1 "$n"); do
  t="$(curl -s -o /dev/null -w '%{time_total} %{http_code}' "$base$url" || echo '0 000')"
  echo "${t% *}" >> "$ttmp"; [ "${t#* }" = "200" ] && ok=$((ok+1)) || fail=$((fail+1))
done
sort -n "$ttmp" | awk -v ok="$ok" -v fail="$fail" '
 { v[NR]=$1 } END {
   p95 = v[int(NR*0.95)>0?int(NR*0.95):NR]; p50 = v[int(NR/2)>0?int(NR/2):1];
   printf "requetes=%d (200: %d, autres: %d)\nmediane=%.3fs p95=%.3fs max=%.3fs\n", NR, ok, fail,
p50, p95, v[NR];
   exit (p95 < 0.5 ? 0 : 1)
 }'; rc=$?; rm -f "$ttmp"
[ $rc -eq 0 ] && echo "ENF-01 conforme (p95 < 500 ms)" || echo "ENF-01 NON conforme : mesurer la
requete (general_log) et verifier les index"
exit $rc
```

G2. Mesure des performances côté base - `tests/perf/mesurer_sql.sh`

(101 lignes, copié verbatim depuis le dépôt.)

```
#!/usr/bin/env bash
# MiniShop - mesure des perFORMances **côté base** (ENF-01, ENF-02, CT-01).
#
# Pourquoi ce script : la mesure HTTP (`tests/perf/mesurer.sh`) exige une application PHP
# et un serveur web. Celle-ci ne mesure QUE la base, sur la volumétrie retenue avec le
# client (200 produits / 1 000 commandes par an, générée par `scripts/gen_volumes.sh`),
# et elle est donc rejouable en soutenance même si l'application n'est pas démarrée.
# Elle répond à trois questions :
#   1. les requêtes types utilisent-elles un index (EXPLAIN, 0 `filesort` sur les filtres) ;
```

```
#   2. la recherche paginée tient-elle sous 500 ms au p95 ;
#   3. un `CALL sp_add_order_line` (verrou + triggers + port) reste-t-il mesurable.
#
# Sortie : tableau des mesures + verdict, et le même bloc copiable dans le doc de tests.
# Usage : DB=minishop_perf ./tests/perf/mesurer_sql.sh [N iterations]
set -uo pipefail
cd "$(dirname "$0")/../.."
DB="${DB:-minishop_perf}"
N="${1:-100}"
MYSQL="$(command -v mysql || command -v mariadb)"
AUTH=()
if [ -n "${DB_USER:-}" ]; then AUTH=(-u"$DB_USER"); [ -n "${DB_PASS:-}" ] && AUTH+=("-p$DB_PASS");
 fi
q() { "$MYSQL" "${AUTH[@]}" "$DB" -N -B "$@"; }

echo "== MiniShop - mesure base '$DB' ($N itérations par requête) =="
q -e "SELECT 'produits', COUNT(*) FROM produit
      UNION ALL SELECT 'commandes', COUNT(*) FROM commande
      UNION ALL SELECT 'lignes', COUNT(*) FROM ligne_commande
      UNION ALL SELECT 'traces_statut', COUNT(*) FROM order_status_history;" |
  while read -r k v; do printf '   %-20s %s\n' "$k" "$v"; done

# --- 1) plans d'exécution ------------------------------------------------
echo
echo "-- EXPLAIN des 4 requêtes types (attendu : pas d'ALL sur la table filtrée) --"
for name in "catalogue paginé" "recherche LIKE" "mes commandes" "commandes par statut"; do
  case "$name" in
    "catalogue paginé")
      sql="SELECT id_produit, reference, nom, prix_ttc FROM v_catalogue ORDER BY id_produit DESC
LIMIT 12 OFFSET 0";;
    "recherche LIKE")
      sql="SELECT id_produit, nom, prix_ttc FROM produit WHERE visible=1 AND (nom LIKE '%écran%'
OR description LIKE '%écran%') ORDER BY nom LIMIT 12";;
    "mes commandes")
      sql="SELECT numero, statut, montant_total, frais_port, date_commande FROM commande WHERE
id_client=1 ORDER BY date_commande DESC LIMIT 10";;
    "commandes par statut")
      sql="SELECT statut, COUNT(*) AS nb, ROUND(AVG(montant_total),2) AS panier_moyen FROM
commande WHERE statut IN ('EN_PREPARATION','PAYEE','EXPEDIEE') GROUP BY statut";;
  esac
  plan=$(q -e "EXPLAIN $sql" | awk -F"\t" '{
            k = ($6=="" || $6=="NULL") ? "aucun index" : $6
            printf "select=%s table=%s type=%s key=%s rows=%s extra=%s", $2, $3, ($4==""?"?":$4),
k, ($9==""?"?":$9), $10
            printf "\n"}' | paste -sd" | " -)
  scan=$(q -e "EXPLAIN $sql" | awk -F"\t" '$4=="ALL"{n++} END{print n+0}')
  if [ "${scan:-0}" -gt 0 ]; then verdict="ACCES COMPLET (ALL) sur $scan table(s) - à documenter";
 else verdict="pas de balayage complet"; fi
  printf '   %-22s %s\n' "$name" "${plan:--}"
  printf '   %-22s ↳ %s\n' '' "$verdict"
done

# --- 2) temps de réponse -------------------------------------------------
mesure() {   # $1 libellé  $2 requête -> temps MOYEN par exécution, en ms (horloge shell)
  local label="$1" sql="$2"
  local t1 t2 dur
```

```
  t1=$(date +%s%N)
  for i in $(seq 1 "$N"); do q -e "$sql" > /dev/null; done
  t2=$(date +%s%N)
  dur=$(( (t2 - t1) / 1000000 / N ))
  printf '%s\t%s\n' "$label" "$dur"
}
echo
echo "-- Temps moyen par requête ($N exécutions, client et serveur sur la même machine) --"
res=$(
  mesure "catalogue (v_catalogue, 12 lignes)" "SELECT id_produit, reference, nom, prix_ttc FROM
v_catalogue ORDER BY id_produit DESC LIMIT 12"
  mesure "recherche plein texte LIKE"         "SELECT id_produit, nom, prix_ttc FROM produit WHERE
 visible=1 AND (nom LIKE '%écran%' OR description LIKE '%écran%') ORDER BY nom LIMIT 12"
  mesure "mes 10 dernières commandes"          "SELECT numero, statut, montant_total, frais_port
FROM commande WHERE id_client=1 ORDER BY date_commande DESC LIMIT 10"
  mesure "indicateurs du jour (GROUP BY)"      "SELECT statut, COUNT(*),
ROUND(AVG(montant_total),2) FROM commande GROUP BY statut"
  mesure "vue v_etat_stock (1 page)"           "SELECT * FROM v_etat_stock WHERE etat <> 'OK'
LIMIT 25"
)
printf '%s\n' "$res" | while IFS=$'\t' read -r label dur; do
  etat="OK"; [ "${dur:-9999}" -gt 500 ] && etat="> 500 ms (ENF-01 non tenu)"
  printf '   %-38s %5s ms   %s\n' "$label" "$dur" "$etat"
done

# --- 3) coût d'une écriture métier ----------------------------------------
echo
echo "-- Écriture métier : sp_add_order_line (verrou FOR UPDATE + triggers + recalcul du port) --"
pid=$(q -e "SELECT MIN(id_produit) FROM produit WHERE stock > 5")
if [ -n "${pid:-}" ]; then
  t1=$(date +%s%N)
  q > /dev/null 2>&1 <<SQL
CALL sp_create_order(1, '12 rue de France, 06000 Nice', @c, @n);
CALL sp_add_order_line(@c, $pid, 1, @l, @t, @r);
CALL sp_confirm_order(@c, 1, 0, @m, @code);
SQL
  t2=$(date +%s%N)
  printf '   commande complète (create + 1 ligne + confirm) : %s ms\n' "$(( (t2 - t1) / 1000000
))"
  q -e "SELECT CONCAT('   -> numéro ', numero, ' - montant ', montant_total, ' - port ',
frais_port)
          FROM commande ORDER BY id_commande DESC LIMIT 1;"
  q -e "SET FOREIGN_KEY_CHECKS=0; DELETE FROM ligne_commande WHERE id_commande = (SELECT
MAX(id_commande) FROM commande); DELETE FROM order_status_history WHERE order_id = (SELECT
MAX(id_commande) FROM commande); DELETE FROM commande WHERE id_commande = (SELECT MAX(id_commande)
 FROM commande); SET FOREIGN_KEY_CHECKS=1;" 2>/dev/null || true
  echo "   (la commande de mesure est supprimée : la base reste dans son état mesuré)"
else
  echo "   aucun produit avec du stock : relancer ./scripts/gen_volumes.sh --purge"
fi
echo
echo "Rappel : ces chiffres mesurent la BASE. La mesure ENF-01 « page < 500 ms (p95) »"
echo "s'obtient avec tests/perf/mesurer.sh une fois l'application déployée (§15.3 du CDC)."
```

H. Générateur de volumétrie - `scripts/gen_volumes.sh`

(104 lignes, copié verbatim depuis le dépôt.)

```
#!/usr/bin/env bash
# MiniShop - generateur de volumetrie pour les mesures de performance (ENF-01 / ENF-02).
#
# Pourquoi ce script existe : un objectif de performance pose sur le jeu de demonstration
# (12 produits, 3 commandes) n'a aucune valeur de mesure. La cible retenue au CDC est
# « petit e-commerce reel » : 200 produits, 1 000 commandes sur 12 mois, 10 clients.
#
# Les commandes sont creees EN PASSANT PAR LES PROCEDURES STOCKEES : le generateur exerce
# donc les memes chemins que l'application (snapshot de prix, frais de port, decrement de
# stock, historique) et les mesures portent sur des donnees coherentes avec RB-06/RB-15.
# Les produits sont inseres par INSERT ... VALUES, JAMAIS par INSERT ... SELECT : verifie
# sur MariaDB 11.8, un INSERT ... SELECT ne declenche pas les triggers BEFORE INSERT qui
# affectent NEW, donc `prix_ttc` (NOT NULL, calcule par trg_produit_ttc) serait refuse.
#
# Usage :
#   bash "$DIR_SCRIPTS/gen_volumes.sh"                          # dans minishop_perf (creee si
besoin)
#   DB=minishop N_PRODUITS=500 N_COMMANDES=2000 bash "$DIR_SCRIPTS/gen_volumes.sh"
#   bash "$DIR_SCRIPTS/gen_volumes.sh" --purge                  # revient au seul jeu de
demonstration
set -uo pipefail
DIR_SCRIPTS="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cd "$(dirname "$0")/.."
DB="${DB:-minishop_perf}"
N_PRODUITS="${N_PRODUITS:-200}"
N_COMMANDES="${N_COMMANDES:-1000}"
N_CLIENTS="${N_CLIENTS:-10}"
BATCH="${BATCH:-25}"
MYSQL="$(command -v mysql || command -v mariadb)"
AUTH=()
if [ -n "${DB_USER:-}" ]; then AUTH=(-u"$DB_USER"); [ -n "${DB_PASS:-}" ] && AUTH+=("-p$DB_PASS");
 fi
run()  { "$MYSQL" "${AUTH[@]}" "$DB" "$@"; }
file() { "$MYSQL" "${AUTH[@]}" "$DB" < "$1"; }

if [ "${1:-}" = "--purge" ]; then
  echo "Purge des donnees generees (produits VP-*, clients perf*, commandes id > 100)."
  run -e "SET FOREIGN_KEY_CHECKS=0;
          DELETE FROM ligne_commande        WHERE id_commande > 100;
          DELETE FROM order_status_history  WHERE order_id    > 100;
          DELETE FROM commande              WHERE id_commande > 100;
          DELETE FROM produit               WHERE reference LIKE 'VP-%';
          DELETE FROM client                WHERE email LIKE 'perf%@minishop.local';
          SET FOREIGN_KEY_CHECKS=1;"
  echo "OK - base ramenee au jeu de demonstration."
  exit 0
fi

echo "== generation dans '$DB' : $N_PRODUITS produits, $N_COMMANDES commandes, $N_CLIENTS clients
=="
run -e "SELECT (SELECT COUNT(*) FROM produit) AS produits_avant, (SELECT COUNT(*) FROM commande)
AS commandes_avant;"

tmp=$(mktemp)
```

```
# --- 1) produits ----------------------------------------------------------
{ echo "INSERT INTO produit (reference, nom, slug, description, prix_ht, tva, stock, seuil_alerte,
 id_categorie, visible) VALUES";
  for i in $(seq 1 "$N_PRODUITS"); do
    prix=$(awk  -v i="$i" 'BEGIN{printf "%.2f", 5 + (i*37 % 900) + (i%10)/10}')
    stk=$(awk   -v i="$i" 'BEGIN{print 20 + (i*7 % 180)}')
    cid=$(awk   -v i="$i" 'BEGIN{print 1 + (i % 4)}')
    sep=$([ "$i" -lt "$N_PRODUITS" ] && echo "," || echo "")
    printf "('%s','Produit de mesure %d','produit-de-mesure-%d','Genere par scripts/gen_volumes.sh
 pour les mesures ENF-01 : description de taille moyenne, prix variable.',%s,20.00,%s,5,%s,1)%s\n"
 \
      "$(printf 'VP-%04d' "$i")" "$i" "$i" "$prix" "$stk" "$cid" "$sep"
  done; } > "$tmp"
file "$tmp" || { echo "ECHEC insertion produits"; exit 1; }

# --- 2) clients -----------------------------------------------------------
{ echo "INSERT IGNORE INTO client (nom, prenom, email, mot_de_passe_hash, adresse_livraison,
code_postal, ville, actif) VALUES";
  for i in $(seq 1 "$N_CLIENTS"); do
    sep=$([ "$i" -lt "$N_CLIENTS" ] && echo "," || echo "")
    printf "('Citoyen-%d','Mesure','perf%d@minishop.local',(SELECT mot_de_passe_hash FROM client
WHERE id_client=1),'%d rue du Benchmark','06000','Nice',1)%s\n" \
      "$i" "$i" "$i" "$sep"
  done; } > "$tmp"
file "$tmp" || { echo "ECHEC insertion clients"; exit 1; }

# --- 3) commandes via les procedures ----------------------------------
nprod=$(run -N -e "SELECT COUNT(*) FROM produit")
ncli=$(run  -N -e "SELECT COUNT(*) FROM client")
t0=$(date +%s)
i=0
while [ "$i" -lt "$N_COMMANDES" ]; do
  { echo "SET @nprod := $nprod; SET @ncli := $ncli;";
    for j in $(seq "$i" $(( i + BATCH - 1 ))); do
      [ "$j" -ge "$N_COMMANDES" ] && break
      cat <<EOF
CALL sp_create_order(1 + ($j % @ncli), CONCAT('Adresse de mesure ', $j, ', 06000 Nice'), @c$j,
@num$j);
CALL sp_add_order_line(@c$j, 1 + ($j % @nprod), 1 + ($j % 3), @l$j, @t$j, @r$j);
CALL sp_add_order_line(@c$j, 1 + (((($j * 7) + 3) % @nprod), 1, @l2$j, @t2$j, @r2$j);
CALL sp_confirm_order(@c$j, 1 + ($j % @ncli), IF($j % 4 = 0, 1, 0), @m$j, @code$j);
EOF
    done; } > "$tmp"
  file "$tmp" || echo "   (lot $i : certaines lignes ont ete refusees par les regles - normal si
un stock est epuise)"
  i=$(( i + BATCH ))
  echo "   $i / $N_COMMANDES commandes - $(( $(date +%s) - t0 )) s"
done
rm -f "$tmp"

echo "== bilan =="
run -e "SELECT (SELECT COUNT(*) FROM produit)             AS produits,
            (SELECT COUNT(*) FROM client)                 AS clients,
            (SELECT COUNT(*) FROM commande)               AS commandes,
            (SELECT COUNT(*) FROM ligne_commande)         AS lignes,
```

```
            (SELECT COUNT(*) FROM order_status_history)    AS traces,
            (SELECT COUNT(*) FROM commande WHERE frais_port > 0) AS commandes_avec_port,
            (SELECT COUNT(*) FROM produit WHERE prix_ttc <> ROUND(prix_ht*(1+tva/100),2)) AS
ttc_incoherents,
            (SELECT COUNT(*) FROM commande c WHERE c.statut <> 'BROUILLON' AND c.montant_total <>
               (SELECT COALESCE(SUM(l.total_ligne),0) FROM ligne_commande l WHERE
l.id_commande=c.id_commande) + c.frais_port) AS montants_incoherents;"
echo "Mesures : DB=$DB ./tests/perf/mesurer.sh   |   Retour au jeu de demo : bash
"$DIR_SCRIPTS/gen_volumes.sh" --purge"
```

I. Chaîne d'intégration continue - `.gitlab-ci.yml`

(114 lignes, copié verbatim depuis le dépôt.)

```
# MiniShop - chaîne d'intégration continue (contrainte CT-09 et §14.2 du cahier des charges).
# Chaque job répond à un critère du barème : un pipeline vert prouve que (a) le script SQL est
# rejouable, (b) les 28 tests (règles, vues, frais de port) passent, (c) aucune règle de sécurité
n'a été
# court-circuitée dans le code PHP. Un job qui échoue bloque le merge : la pénalité devient
# visible pendant le développement, pas au moment du rendu.
#
# Prod : GitLab CI (ce fichier) - cible de notation, barème §16.1.
# Dev  : GitHub Actions (.github/workflows/ci.yml) - même 6 jobs adaptés pour GitHub
#        (setup-php, services mysql:8.0, fallback sans load_db.sh). Les deux pipelines
#        sont maintenus synchrones ; en dev on pousse sur GitHub, en prod on miroir vers GitLab.
#
# Règle de re-run (CDC §14.2, annexe 17.9 §7) : si le pipeline est VERT avant le merge
# (statut de la MR « Pipeline passed »), aucun re-run n'est nécessaire après le merge.
# Le pipeline post-merge rejoue les mêmes jobs sur le même arbre de fichiers (GitLab teste
# la MR sur son merge result) : il sert de preuve/badge, pas de contrôle. On ne relance que
# si main a avancé, si le run a été annulé, si le code a été corrigé puis poussé (le push
# relance seul), ou après une panne d'infrastructure (un Retry, anomalie consignée A-0x).

stages: [validate, db, test, docs]

default:
  interruptible: true

# 1. Syntaxe PHP : 100 % des fichiers doivent être valides (§11.3 PRÉ-3).
php-lint:
  stage: validate
  image: php:8.2-cli
  script:
    - find app public -name '*.php' -print0 | xargs -0 -n1 -P4 php -l

# 2. Conventions PSR-12 : signalées, non bloquantes (ne doit jamais faire échouer une démo).
style:
  stage: validate
  image: php:8.2-cli
  allow_failure: true
  script:
    - "command -v phpcs >/dev/null 2>&1 || pear install -f PHP_CodeSniffer || true"
    - "phpcs --standard=PSR12 --report=summary app public || true"
```

```
# 3. Contrôles statiques de sécurité : 10 grep bloquants (interpolation SQL, échappement
#    manquant, mot de passe en dur, session durcie, CSRF, contrôle d'appartenance).
security-scan:
  stage: validate
  image: alpine:3.20
  before_script:
    - apk add --no-cache bash grep findutils
  script:
    - bash tests/security/controles.sh

# 4. Base de données : chargement du schéma puis 28 tests (règles, vues, port).
#    Porte les points « script SQL », « procédures stockées », « triggers » du barème.
base-tests:
  stage: db
  image: mysql:8.0
  services:
    - name: mysql:8.0
      alias: mysql
  variables:
    MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
    MYSQL_DATABASE: minishop_test
    DB_HOST: mysql
    DB_USER: root
    DB_PASS: ""
    DB_TEST: minishop_test
  before_script:
    - apt-get update -qq
    - apt-get install -y -qq --no-install-recommends mariadb-client
  script:
    - ./scripts/load_db.sh
    - ./scripts/run_sql_tests.sh
  artifacts:
    when: always
    expire_in: 4 weeks
    paths:
      - tests/sql/rapport_tests_sql.md
      - tests/sql/out/

# 5. Tests unitaires PHP (modèles, repositories, sécurité) sur la base de test.
unit:
  stage: test
  image: php:8.2-cli
  services:
    - name: mysql:8.0
      alias: mysql
  variables:
    MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
    MYSQL_DATABASE: minishop_test
    DB_TEST: minishop_test
  before_script:
    - apt-get update -qq
    - apt-get install -y -qq --no-install-recommends mariadb-client unzip git
    - "test -f composer.phar || (curl -sS https://getcomposer.org/installer | php)"
    - php composer.phar install --prefer-dist --no-progress
  script:
    - ./vendor/bin/phpunit --testsuite app --log-junit phpunit.xml
```

```
  artifacts:
    when: always
    reports:
      junit: phpunit.xml

# 6. Documentation : les diagrammes doivent se régénérer depuis leurs sources, sinon la
#    pénalité « manque un diagramme UML » (-1) s'applique au rendu.
docs:
  stage: docs
  image: openjdk:17-slim
  before_script:
    - apt-get update -qq && apt-get install -y -qq --no-install-recommends curl || true
    - "test -f plantuml.jar || curl -sSL -o plantuml.jar
https://github.com/plantuml/plantuml/releases/download/v1.2024.0/plantuml-1.2024.0.jar"
  script:
    - PLANTUML_JAR=plantuml.jar bash docs/diagrams/render.sh
  artifacts:
    expire_in: 1 week
    paths:
      - docs/diagrams/
```

Règle de re-run : si le pipeline est **vert avant le merge**, aucun re-run n'est nécessaire après le merge - la MR est testée sur son merge result (GitHub Actions : `refs/pull/<n>/merge`), donc le run post-merge rejoue les mêmes jobs sur le même arbre de fichiers. Exceptions et tableau des six cas : **CDC §14.2** ; application à la charte d'équipe : **annexe 17.9 §7**.

J. Fichiers hors dépôt - `.gitignore`

(35 lignes, copié verbatim depuis le dépôt.)

```
# MiniShop - fichiers volontairement hors dépôt.
# Règle n° 7 du §14.1 du cahier des charges : aucun secret dans GitLab, aucune sortie
# régénérable encombrant l'historique. La ligne « tests/sql/out » est un arbitrage
# documenté : seul le rapport consigné dans le rendu est versionné (c'est la preuve
# demandée), les journaux bruts par test restent locaux.

# --- secrets et configuration locale (le modèle app/Config/env.example.php est, lui, versionné)
app/Config/env.php
config/env.php
.env
.env.local
*.pem
*.key

# --- données locales, journaux applicatifs
var/
!var/.gitkeep
*.log

# --- artefacts générés (reproductibles par les scripts du dépôt)
vendor/
node_modules/
dist/
```

```
docs/diagrams/*.svg.tmp

# --- sorties de tests : régénérées par ./scripts/run_sql_tests.sh
tests/sql/out/*
!tests/sql/rapport_tests_sql.md

# --- éditeurs / OS
.idea/
.vscode/
*.swp
.DS_Store
Thumbs.db
```